

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования

**«Белорусский государственный университет информатики и
радиоэлектроники»**

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

ОТЧЕТ

по лабораторной работе №4

по дисциплине

**«Современные технологии программирования масштабируемых
приложений»**

«Эмулятор банкомата»

Выполнил:

Лебедев Артем Владимирович
магистрант группы 556301

Преподаватель:

Хмелёва Ангелина Владимировна
кандидат технических наук
доцент кафедры ПОИТ

Минск, 2026 г.

СОДЕРЖАНИЕ

1	Цель работы	1
2	Задание	2
3	Проектирование	3
3.1	Архитектура	4
3.2	Протокол общения банка и банкомата	4
3.3	Конечный автомат сеанса	6
3.4	Внутренний список	7
3.5	Операции банка	7
3.6	Защита PIN-кода	7
4	Реализация	8
4.1	Структура	9
4.2	Автомат сеанса и внутренний список	9
4.3	Банковская сторона протокола	11
4.4	Клиентский интерфейс	12
5	Тестирование	12
6	Результаты работы	14
	Заключение	27
	Список использованных источников	28
	Приложение А Исходный код эмулятора банкомата и протокола	29

1 ЦЕЛЬ РАБОТЫ

Разработать эмулятор банкомата, взаимодействующий с виртуальным банком из предыдущих работ: реализовать клиентский интерфейс, повторяющий работу с реальным устройством, и протокол общения банкомата с банком на основе транзакций. Рабочим счётом карты служит кредитный счёт, открытый в третьей лабораторной работе.

2 ЗАДАНИЕ

Банкомат — автоматизированное устройство, которое позволяет держателю счёта удалённо выполнять операции:

- аутентификация по номеру карты и PIN-коду; после трёх неверных PIN-кодов подряд работа с банкоматом принудительно завершается;
- снятие денег с кредитного счёта;
- просмотр остатка кредитного счёта;
- просмотр остатка депозитного счёта (необязательная операция);
- платёж за услуги оператора мобильной связи (необязательная операция);
- возврат карты.

Модель общения с банком построена на транзакциях. В ходе диалога с пользователем банкомат накапливает вводимую информацию во *внутреннем списке*: номер карты, PIN-код, вид операции, её параметры. Когда собраны все данные, необходимые для операции, содержимое списка целиком отправляется серверу банка, а список очищается. Для следующей операции данные карты вводятся заново; номер карты эмулятор запоминает, поскольку он имитирует саму карту, оставшуюся в устройстве.

Операции со счётом сопровождаются чеком: при платёже чек печатается автоматически, в остальных случаях — по желанию клиента. Номер карты и PIN-код вводятся с клавиатуры; сумма снятия должна быть неотрицательным целым числом; номер телефона при платёже состоит из десяти цифр и перед отправкой подтверждается клиентом. В работе реализованы все операции, включая необязательные.

3 ПРОЕКТИРОВАНИЕ

3.1 Архитектура

Задание выделяет в программе две системы — клиентский интерфейс и протокол общения банка и банкомата. Им соответствуют два уровня решения (рисунок 3.1).

Банкомат реализован отдельным микросервисом `atm-service`. Он не имеет базы данных и ничего не знает о счетах и договорах: его задача — вести диалог с клиентом и собирать транзакцию. Web-интерфейс выполнен как «тонкий» терминал: браузер только рисует экран, который описал сервис, и сообщает о нажатиях. Вся логика сеанса находится на сервере, где её можно тестировать без браузера.

Банковскую сторону протокола реализует сервис кредитов — владелец карты и кредитного счёта. Он проверяет карту и PIN-код, проводит операции через главную книгу, а остатки вкладов запрашивает у сервиса депозитов. Банкомату доступна единственная точка входа банка, и получить доступ к счёту в обход проверки PIN-кода он не может.

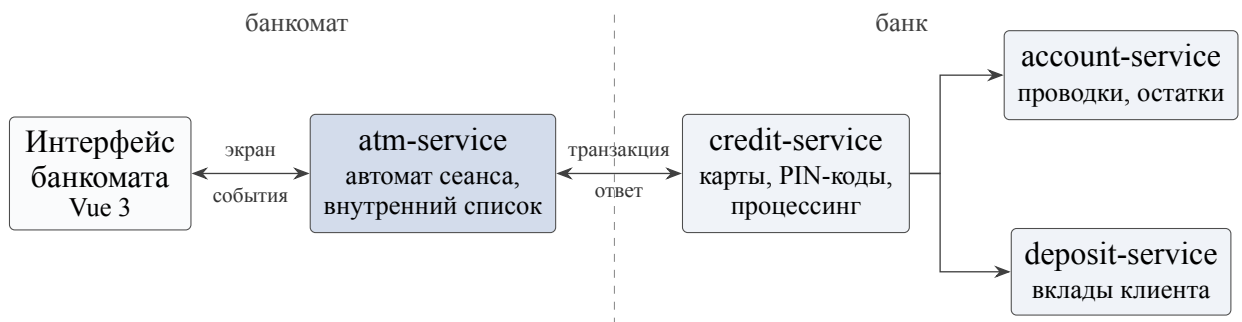


Рисунок 3.1 — Банкомат и банк: уровни решения

3.2 Протокол общения банка и банкомата

Транзакция — упорядоченный список пар «имя — значение», то есть содержимое внутреннего списка банкомата. Она передаётся одним запросом `POST /api/atm/transactions`; элементы списка перечислены в таблице 3.1. Банк отвечает статусом `OK` или `ERROR`, кодом, текстом для экрана и данными для чека (таблица 3.2). Пример обмена приведён в листинге 3.1.

Таблица 3.1 – Элементы транзакции

Имя	Значение	Когда добавляется в список
CARD	номер карты, 16 цифр	после ввода номера; перед новой операцией — автоматически
PIN	PIN-код, 4 цифры	после ввода PIN-кода
OPERATION	AUTHORIZE, WITHDRAW, BALANCE, DEPOSIT_BALANCE, PAYMENT	при выборе пункта меню
AMOUNT	сумма	после ввода суммы снятия или платежа
OPERATOR	код оператора связи	после выбора оператора
PHONE	номер телефона, 10 цифр	после ввода номера

Таблица 3.2 – Коды ответа банка

Код	Значение
OK	операция выполнена; в данных — сумма, остаток, реквизиты для чека
WRONG_PIN	неверный PIN-код; в данных — число оставшихся попыток
CARD_BLOCKED	карта заблокирована после трёх неверных PIN-кодов подряд
CARD_NOT_FOUND	карта не обслуживается банком
INSUFFICIENT_FUNDS	на счёте недостаточно средств
INVALID_AMOUNT, INVALID_PHONE	неверная сумма или номер телефона
UNKNOWN_OPERATOR, UNKNOWN_OPERATION	оператора связи или операции нет в справочнике
SERVICE_UNAVAILABLE	смежный сервис банка временно недоступен

Листинг 3.1 – Транзакция снятия наличных и ответ банка

```
POST /api/atm/transactions
{"items": [
  {"name": "CARD",      "value": "9112380000000036"},
  {"name": "PIN",       "value": "****"},
  {"name": "OPERATION", "value": "WITHDRAW"},
  {"name": "AMOUNT",    "value": "450"}
]}

200 OK
{"status": "OK", "code": "OK", "message": "Заберите деньги",
 "data": {"amount": 450, "balance": 2550.00, "currency": "BYN",
```

```
"bankDate": "2026-11-01", "holder": "Лебедев Артем Владимирович"}}
```

Банк не хранит состояние диалога: каждая транзакция содержит номер карты и PIN-код и проверяется заново. Поэтому банковская сторона остаётся сервисом без состояния и может масштабироваться горизонтально, а состояние сеанса живёт только в банкомате.

3.3 Конечный автомат сеанса

Работа с банкоматом описана конечным автоматом из одиннадцати состояний (рисунок 3.2). События автомата — ввод значения с клавиатуры (клавиша «Ввод»), выбор пункта меню боковой кнопкой и клавиша «Отмена».

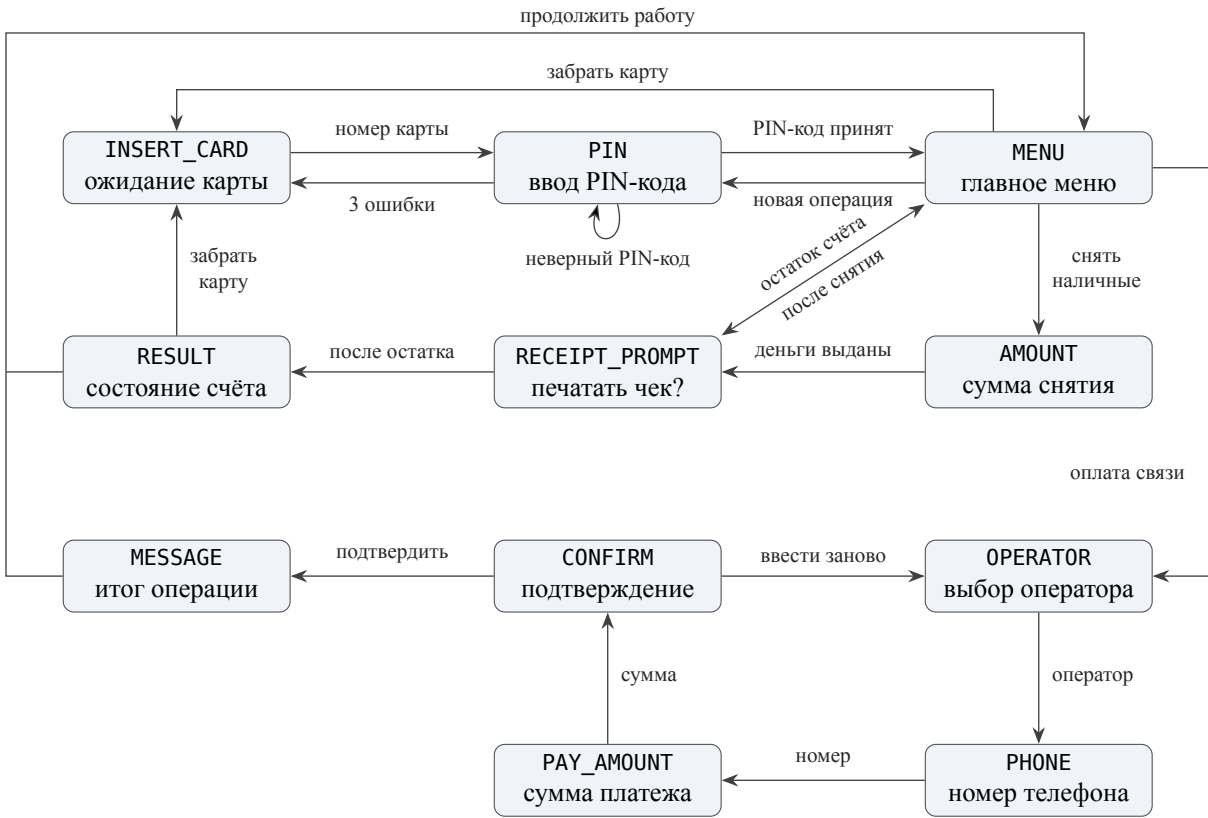


Рисунок 3.2 – Конечный автомат сеанса работы с банкоматом (основные переходы)

На рисунке не показаны переходы по отказу банка: из состояний AMOUNT и CONFIRM, а также при запросе остатка отказ (например, нехватка средств) ведёт в состояние MESSAGE, откуда клиент возвращается в меню. Клавиша «Отмена» внутри операции возвращает в меню, а в меню и при вводе PIN-кода — возвращает карту.

3.4 Внутренний список

Жизненный цикл внутреннего списка на примере двух операций подряд показан в таблице 3.3. После авторизации номер карты и PIN-код остаются в списке и используются первой операцией. Отправка транзакции очищает список, поэтому перед второй операцией банкомат сам добавляет номер карты, запрашивает PIN-код и повторно авторизует карту — неверный PIN-код обнаруживается сразу, а не после ввода суммы.

Таблица 3.3 – Внутренний список при снятии наличных и последующем запросе остатка

Действие клиента	Содержимое списка	Обращение к банку
Ввод номера карты	CARD	—
Ввод PIN-кода	CARD, PIN	AUTHORIZE
Выбор «Снять наличные»	CARD, PIN, OPERATION	—
Ввод суммы	CARD, PIN, OPERATION, AMOUNT	WITHDRAW
Ответ банка получен	пуст	—
Выбор «Остаток счёта»	CARD (подставлен банкоматом)	—
Ввод PIN-кода	CARD, PIN	AUTHORIZE
	CARD, PIN, OPERATION	BALANCE
Ответ банка получен	пуст	—

3.5 Операции банка

Снятие наличных в банкомате — это операции 2 и 3 схемы «Кредитная программа»: «Перевод кредита в кассу» (кредит текущего счёта, дебет кассы) и «Получение кредита через кассу» (кредит кассы). Перед проводкой сервис кредитов сравнивает сумму с остатком текущего счёта; тот же запрет отрицательного остатка действует и в главной книге.

Платёж за услуги связи списывает сумму с текущего счёта клиента и зачисляет её на расчётный счёт оператора — пассивный счёт 3012 «Текущие (расчётные) счета коммерческих организаций», который открывается в главной книге при первом платеже этому оператору.

Запрос остатка депозитного счёта сервис кредитов переадресует сервису депозитов и возвращает действующие вклады держателя карты.

3.6 Защита PIN-кода

PIN-код генерируется при выпуске карты случайным образом и показывается сотруднику банка один раз («ПИН-конверт»). В базе хранится только его хеш BCrypt, восстановить PIN-код по которому нельзя; при утрате он перераспечатывается. Счётчик неверных попыток ведёт банк, а не банкомат: после третьей ошибки подряд карта блокируется, и начать подбор заново в новом

сеансе или на другом банкомате невозможно. Блокировка снимается перепрограммированием PIN-кода. На экране, во внутреннем списке и в журнале обмена PIN-код отображается звёздочками.

Номер карты содержит контрольную цифру по алгоритму Луна. Банкомат проверяет её сам и не обращается в банк, если номер набран с ошибкой.

4 РЕАЛИЗАЦИЯ

4.1 Структура

Состав модуля банкомата и банковской части протокола приведён в листинге 4.1.

Листинг 4.1 – Модуль atm-service и пакет протокола в credit-service

```
atm-service/  
  bank/      BankGateway – REST-клиент банка; Transaction, Item, BankReply  
  session/    AtmSession – конечный автомат сеанса и внутренний список  
              AtmState, Screen, Receipts, AtmService (реестр сеансов)  
  web/        AtmController – REST клиентского интерфейса  
  resources/  static/index.html, app.js, atm.css – интерфейс банкомата  
credit-service/  
  atm/        AtmProcessor – обработка транзакций; AtmRequest, AtmResponse, DepositApi  
  domain/     Card, CardNumbers, MobileOperator, MobilePayment  
  service/    CardService – выпуск карты, проверка и перевыпуск PIN-кода  
  web/        AtmController – POST /api/atm/transactions
```

4.2 Автомат сеанса и внутренний список

Сеанс представлен объектом AtmSession. Событие «Ввод» обрабатывается в зависимости от текущего состояния (листинг 4.2): значение проверяется, добавляется во внутренний список buffer, и автомат переходит в следующее состояние либо обращается к банку.

Листинг 4.2 – Обработка ввода с клавиатуры (AtmSession.enter)

```
public synchronized Screen enter(String text) {  
    begin();  
    String value = text == null ? "" : text.replace(" ", "");  
    switch (state) {  
        case INSERT_CARD -> insertCard(value);  
        case PIN -> {  
            if (!value.matches("\\d{4}")) {  
                notice = "PIN-код состоит из 4 цифр";  
            } else {  
                buffer.add(new Item(Item.PIN, value));  
                authorize();  
            }  
        }  
        case AMOUNT -> {  
            if (acceptAmount(value)) {  
                execute();  
            }  
        }  
        case PHONE -> {  
            if (!value.matches("\\d{10}")) {  
                notice = "Номер телефона – 10 цифр, например 0291234567";  
            } else {  
                buffer.add(new Item(Item.PHONE, value));  
                state = AtmState.PAY_AMOUNT;  
            }  
        }  
    }  
}
```

```

        case PAY_AMOUNT -> {
            if (acceptAmount(value)) {
                state = AtmState.CONFIRM;
            }
        }
        default -> { }
    }
    return screen();
}

```

Отправка транзакции и очистка списка сосредоточены в одном методе (листинг 4.3). По ответу банка автомат выбирает следующее состояние: при платеже чек печатается сразу, при снятии и запросе остатка клиенту задаётся вопрос о чеке, при отказе показывается сообщение банка.

Листинг 4.3 – Отправка транзакции банку (AtmSession.execute)

```

/** Отправка накопленной транзакции: содержимое списка уходит банку целиком, список очищается.
 */
private void execute() {
    reply = send(new ArrayList<>(buffer));
    buffer.clear();
    if (reply == null) {
        state = AtmState.MESSAGE;
        notice = "Нет связи с банком. Операция не выполнена";
    } else if (!reply.ok()) {
        if ("WRONG_PIN".equals(reply.code()) || "CARD_BLOCKED".equals(reply.code())) {
            eject(reply.message() + ". Заберите карту");
        } else {
            state = AtmState.MESSAGE;
            notice = reply.message();
        }
    } else if ("PAYMENT".equals(operation)) {
        receipt = Receipts.print(operation, reply, card, terminalId, clock); // чек
        платежа печатается всегда
        state = AtmState.MESSAGE;
    } else {
        if ("WITHDRAW".equals(operation)) {
            cash = Receipts.money(reply, "amount");
        }
        state = AtmState.RECEIPT_PROMPT;
    }
}

```

Правило «данные карты вводятся заново» реализовано при выборе операции (листинг 4.4): если в списке нет номера карты и PIN-кода, банкомат подставляет запомненный номер и переходит к вводу PIN-кода, запомнив выбранную операцию.

Листинг 4.4 – Повторный ввод данных карты (AtmSession.chooseOperation)

```

private void chooseOperation(String option) {
    if (hasCredentials()) {
        startOperation(option);
    } else {
        buffer.clear();
        buffer.add(new Item(Item.CARD, card));
    }
}

```

```

        pendingOperation = option;
        state = AtmState.PIN;
    }
}

```

4.3 Банковская сторона протокола

Транзакцию принимает класс `AtmProcessor` сервиса кредитов (листинг 4.5). Проверка карты и PIN-кода выполняется для каждой транзакции до разбора операции. Отказы возвращаются не исключениями, а обычным ответом протокола: это важно для счётчика неверных попыток — он изменяется в той же транзакции базы данных и должен быть сохранён.

Листинг 4.5 – Проверка карты и PIN-кода, выбор операции (`AtmProcessor.handle`)

```

Card card = cards.findByNumber(tx.get("CARD")).orElse(null);
if (card == null) {
    return AtmResponse.error("CARD_NOT_FOUND", "Карта не обслуживается банком");
}
if (card.isBlocked()) {
    return AtmResponse.error("CARD_BLOCKED", "Карта заблокирована. Обратитесь в банк");
}
if (!cardService.matches(card, tx.get("PIN"))) {
    int left = card.registerFailedAttempt();
    return left == 0
        ? AtmResponse.error("CARD_BLOCKED", "PIN-код трижды введён неверно. Карта заблокирована")
        : AtmResponse.error("WRONG_PIN", "Неверный PIN-код. Осталось попыток: " + left,
            Map.of("attemptsLeft", left));
}
card.setFailedAttempts(0);

try {
    return switch (operation) {
        case "AUTHORIZE" -> AtmResponse.ok("Авторизация выполнена", Map.of("holder", card.getHolder()));
        case "BALANCE" -> balance(card);
        case "DEPOSIT_BALANCE" -> depositBalance(card);
        case "WITHDRAW" -> withdraw(card, tx.get("AMOUNT"));
        case "PAYMENT" -> payment(card, tx.get("OPERATOR"), tx.get("PHONE"), tx.get("AMOUNT"));
        ;
        default -> AtmResponse.error("UNKNOWN_OPERATION", "Операция не поддерживается");
    };
} catch (BankException e) {
    return AtmResponse.error(e.getCode(), e.getMessage());
} ...

```

Банк не доверяет проверкам банкомата и повторяет их: сумма снятия должна быть целым положительным числом, номер телефона — десятью цифрами с кодом оператора 25, 29, 33 или 44, оператор — записью справочника.

4.4 Клиентский интерфейс

Интерфейс повторяет лицевую панель банкомата: экран, по четыре боковые кнопки с каждой стороны, цифровая клавиатура с клавишами «Отмена», «Сброс» и «Ввод», приёмник карты, лотки выдачи денег и чека. Сервис возвращает описание экрана — заголовок, строки текста, вид ожидаемого ввода, пункты меню, выданные деньги, строки чека; интерфейс отображает его и отправляет одно из трёх событий (таблица 4.1). Пункты меню привязаны к боковым кнопкам. Пока идёт обращение к банку, на экране показывается надпись «Авторизация на сервере...». Цифры можно набирать и с клавиатуры компьютера.

Справа от банкомата для наглядности выводятся текущее состояние автомата, содержимое внутреннего списка и последняя транзакция с ответом банка.

Таблица 4.1 – REST клиентского интерфейса банкомата

Метод	Адрес	Назначение
POST	/api/sessions	начало сеанса; в ответе — идентификатор и первый экран
GET	/api/sessions/{id}	текущий экран
POST	/api/sessions/{id}/enter	клавиша «Ввод» с набранным значением
POST	/api/sessions/{id}/select	боковая кнопка: выбран пункт меню
POST	/api/sessions/{id}/cancel	клавиша «Отмена»

5 ТЕСТИРОВАНИЕ

Банкомат и банковская сторона протокола покрыты 68 тестами (таблица 5.1). Автомат сеанса тестируется как обычный объект с заглушкой банка: тесты подают события и проверяют экран, содержимое внутреннего списка и транзакцию, отправленную банку (листинг 5.1). Функциональные тесты проходят сценарии в браузере через WebDriver. Итог прогона приведён в листинге 5.2.

Таблица 5.1 – Тесты эмулятора банкомата и протокола

Класс тестов	Тестов	Что проверяется
<i>atm-service</i>		
AtmSessionTest	27	переходы автомата, внутренний список, чеки, отказы
AtmApiIT	3	REST интерфейса, независимость сеансов
AtmUiIT	3	снятие с чеком, три неверных PIN-кода, платёж — в браузере
<i>credit-service</i>		
AtmProtocolIT	25	все операции протокола, блокировка карты, проводки
CardNumbersTest	10	алгоритм Луна, счётчик попыток
Всего	68	

Листинг 5.1 – Тест правила повторного ввода PIN-кода (AtmSessionTest)

```
@Test
@DisplayName("Перед следующей операцией PIN-код вводится заново, номер карты банкомат помнит сам")
void secondOperationRequiresPinAgain() {
    login(atm);
    atm.select("WITHDRAW");
    atm.enter("100");
    atm.select("NO");

    Screen pin = atm.select("BALANCE");
    assertThat(pin.state()).isEqualTo("PIN");
    assertThat(pin.title()).isEqualTo("Введите PIN-код ещё раз");
    assertThat(pin.buffer()).containsExactly(new Item("CARD", CARD));

    Screen prompt = atm.enter(PIN);
    assertThat(prompt.state()).isEqualTo("RECEIPT_PROMPT");
    assertThat(sent().items()).containsExactly(new Item("CARD", CARD), new Item("PIN", PIN),
        new Item("OPERATION", "BALANCE"));
}
```

Листинг 5.2 – Результат выполнения тестов

```
$ ./mvnw -pl atm-service,credit-service -am verify
```

```
Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.055 s -- in by.bsuir.bank.credit.CardNumbersTest
```

```
Tests run: 25, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.605 s -- in by.bsuir.bank.credit.AtmProtocolIT
```

```
Tests run: 27, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.446 s -- in by.bsuir.bank.atm.AtmSessionTest
```

```
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 9.761 s -- in by.bsuir.bank.atm.AtmUiIT
```

```
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.162 s -- in by.bsuir.bank.atm.AtmApiIT
```

```
Tests run: 68, Failures: 0, Errors: 0, Skipped: 0
```

```
BUILD SUCCESS
```

6 РЕЗУЛЬТАТЫ РАБОТЫ

Для демонстрации клиенту выдан кредит 3 000 BYN с картой. Все пять сервисов запущены, банкомат открыт по адресу <http://localhost:8085>.

Сеанс начинается с ввода номера карты (рисунок 6.1); номер отображается группами по четыре цифры. После неверного PIN-кода банкомат сообщает число оставшихся попыток и запрашивает его повторно (рисунок 6.2); в панели справа виден ответ банка с кодом `WRONG_PIN`.

The screenshot displays the web interface of an ATM from 'Bank «Решение»'. The top navigation bar includes links for 'Клиенты', 'Счета', 'Депозиты', 'Кредиты', and 'Банкомат'. The main interface is divided into two sections. The left section, titled 'Банкомат 24 ЧАСА', shows a large screen with the instruction 'Вставьте, пожалуйста, карту (введите её номер)' and the card number '9112 3800 0000 0036' displayed in groups of four. Below the screen is a numeric keypad with buttons for digits 1-9, 0, and function buttons 'ОТМЕНА' (red), 'СБРОС' (yellow), and 'ВВОД' (green). To the right of the keypad are two horizontal bars labeled 'ВЫДАЧА ДЕНЕГ' and 'ЧЕК'. The right section of the interface contains two informational boxes. The first, 'Состояние автомата', shows 'INSERT_CARD' and explains that numbers can be entered via the keypad or keyboard (Enter for 'Ввод', Esc for 'Отмена'). The second box, 'Внутренний список банкомата', shows 'пуст' (empty).

Рисунок 6.1 – Ввод номера карты

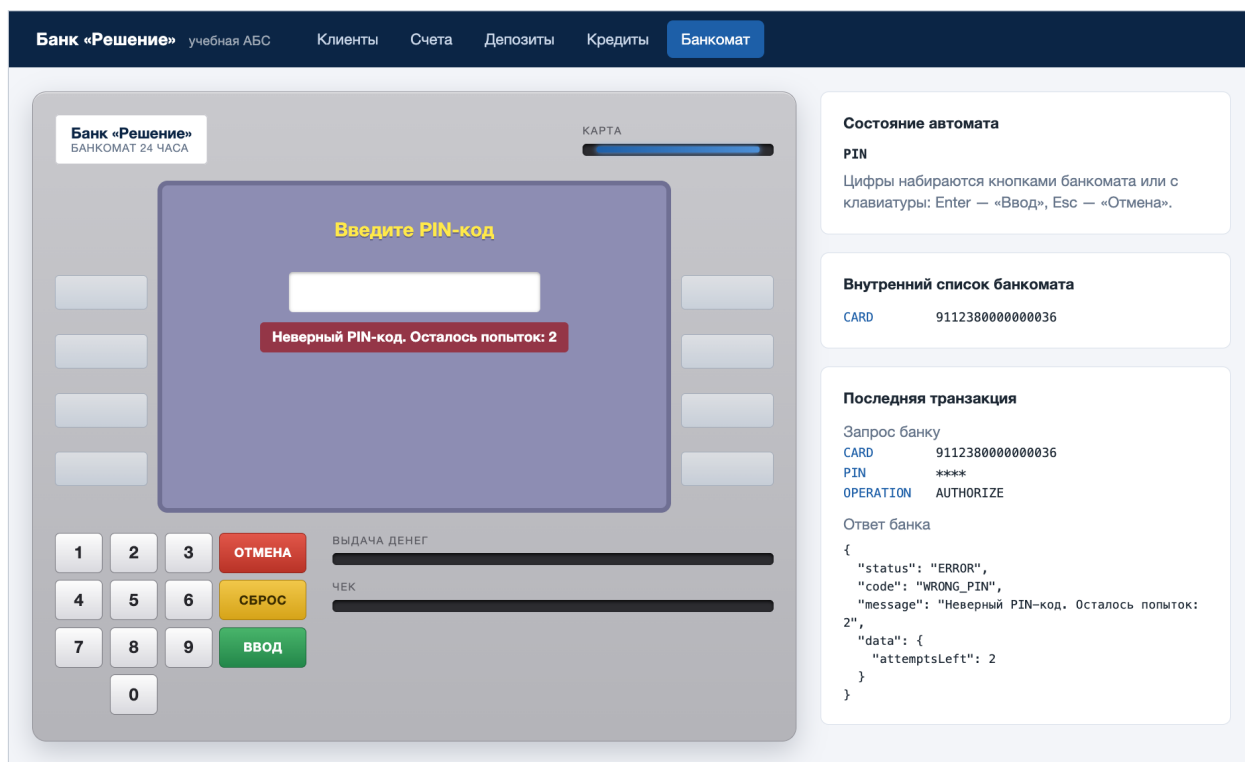


Рисунок 6.2 – Неверный PIN-код: повторный запрос

После верного PIN-кода открывается главное меню (рисунок 6.3). Во внутреннем списке остаются номер карты и скрытый PIN-код, в журнале обмена — транзакция авторизации.

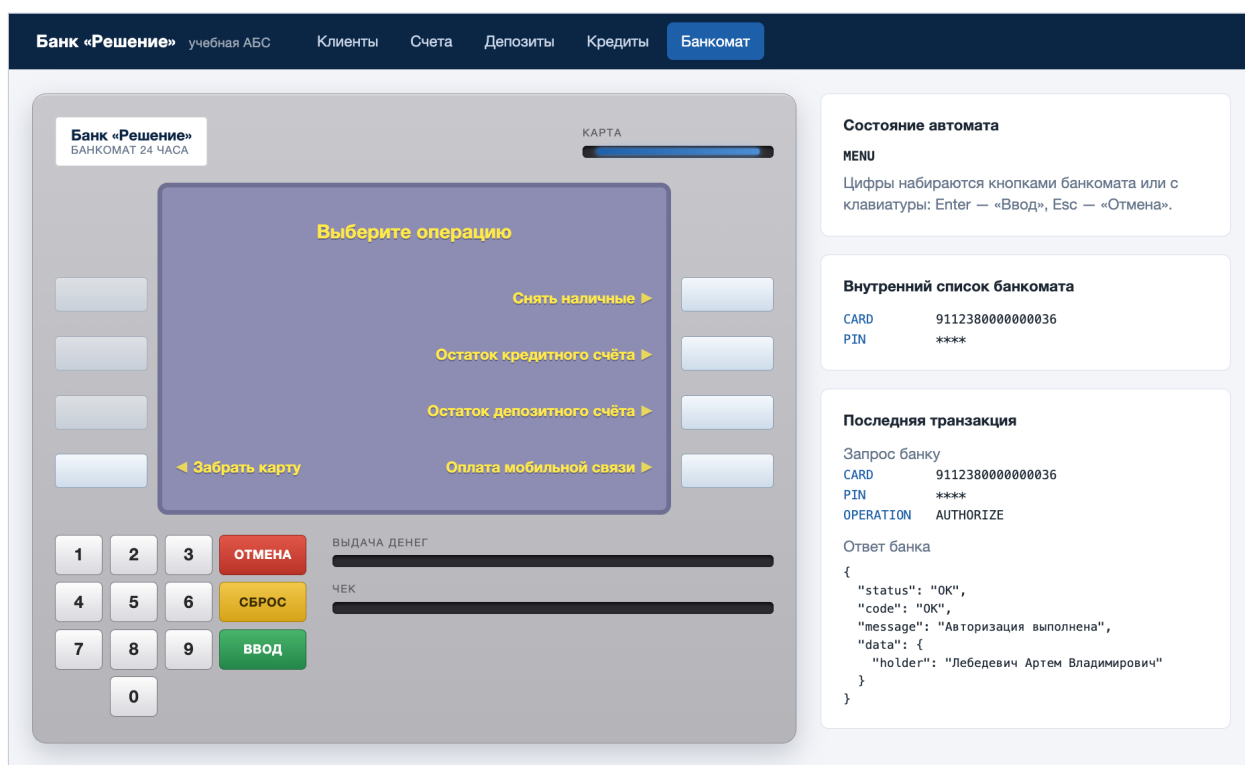


Рисунок 6.3 – Главное меню

Снятие наличных показано на рисунках 6.4–6.6: клиент вводит сумму,

банк подтверждает операцию, деньги появляются в лотке выдачи, и по желанию клиента печатается чек с суммой и остатком счёта. После отправки транзакции внутренний список пуст.

Банк «Решение» учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Банк «Решение»
БАНКОМАТ 24 ЧАСА

КАРТА

Введите сумму
Снятие наличных с кредитного счёта, RUB

450

1234567890

ОТМЕНА

СБРОС

ВВОД

ВЫДАЧА ДЕНЕГ

ЧЕК

Состояние автомата

AMOUNT

Цифры набираются кнопками банкомата или с клавиатуры: Enter — «Ввод», Esc — «Отмена».

Внутренний список банкомата

CARD 9112380000000036

PIN ****

OPERATION WITHDRAW

Последняя транзакция

Запрос банку

CARD 9112380000000036

PIN ****

OPERATION AUTHORIZE

Ответ банка

{
 "status": "OK",
 "code": "OK",
 "message": "Авторизация выполнена",
 "data": {
 "holder": "Лебедев Артем Владимирович"
 }
}

Рисунок 6.4 – Ввод суммы снятия

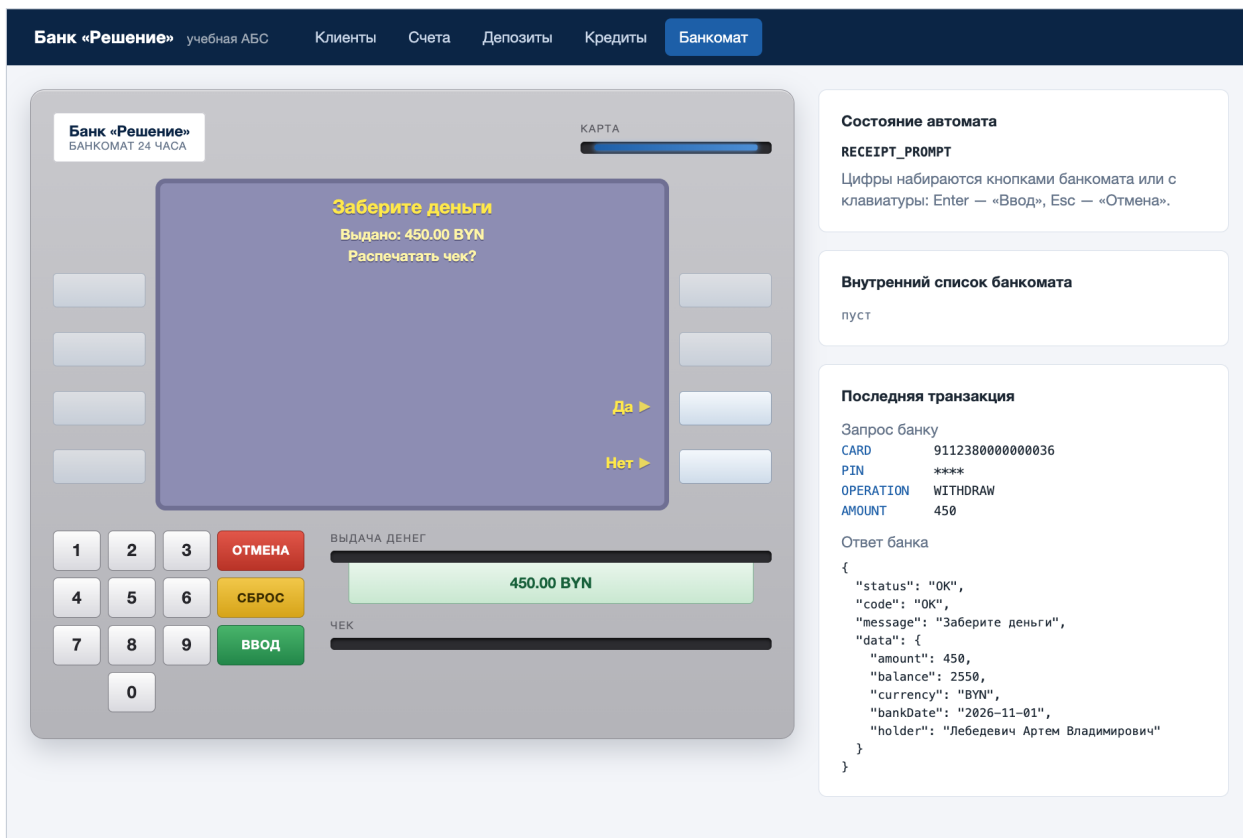


Рисунок 6.5 – Деньги выданы, вопрос о печати чека

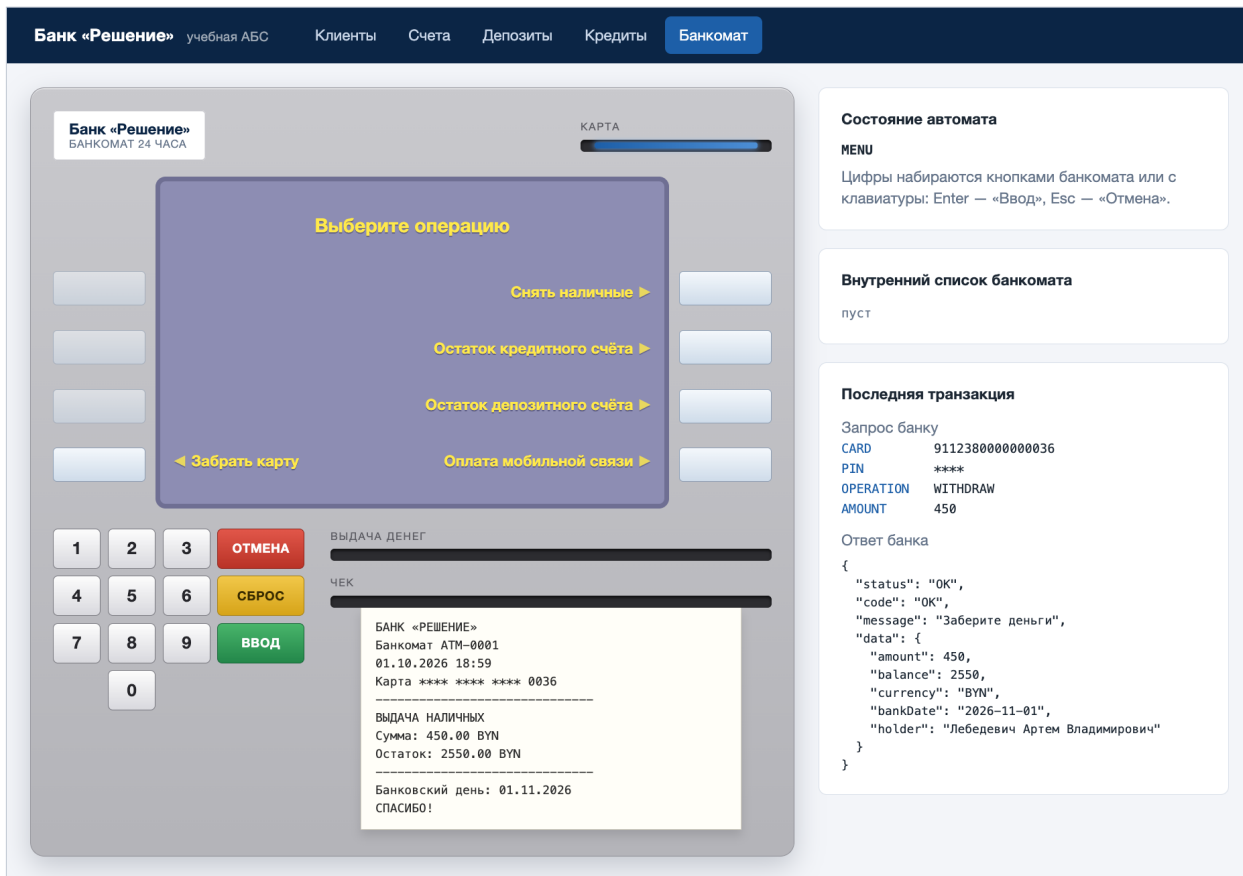


Рисунок 6.6 – Чек о выдаче наличных

Перед следующей операцией банкомат запрашивает PIN-код ещё раз: номер карты уже подставлен в список (рисунок 6.7). Остаток кредитного счёта выводится на экран и в чек; на этом шаге клиент может продолжить работу или забрать карту (рисунок 6.8).

Банк «Решение» учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Банк «Решение»
БАНКОМАТ 24 ЧАСА

KAPTA

Введите PIN-код ещё раз
Для новой операции данные карты вводятся заново

1

2

3

ОТМЕНА

4

5

6

СБРОС

7

8

9

ВВОД

0

ВЫДАЧА ДЕНЕГ

ЧЕК

Состояние автомата

PIN

Цифры набираются кнопками банкомата или с клавиатуры: Enter — «Ввод», Esc — «Отмена».

Внутренний список банкомата

CARD 9112380000000036

Последняя транзакция

Запрос банку

CARD 9112380000000036

PIN ****

OPERATION WITHDRAW

AMOUNT 450

Ответ банка

{
 "status": "OK",
 "code": "OK",
 "message": "Заберите деньги",
 "data": {
 "amount": 450,
 "balance": 2550,
 "currency": "BYN",
 "bankDate": "2026-11-01",
 "holder": "Лебедев Артем Владимирович"
 }
}

Рисунок 6.7 – Повторный ввод PIN-кода перед новой операцией

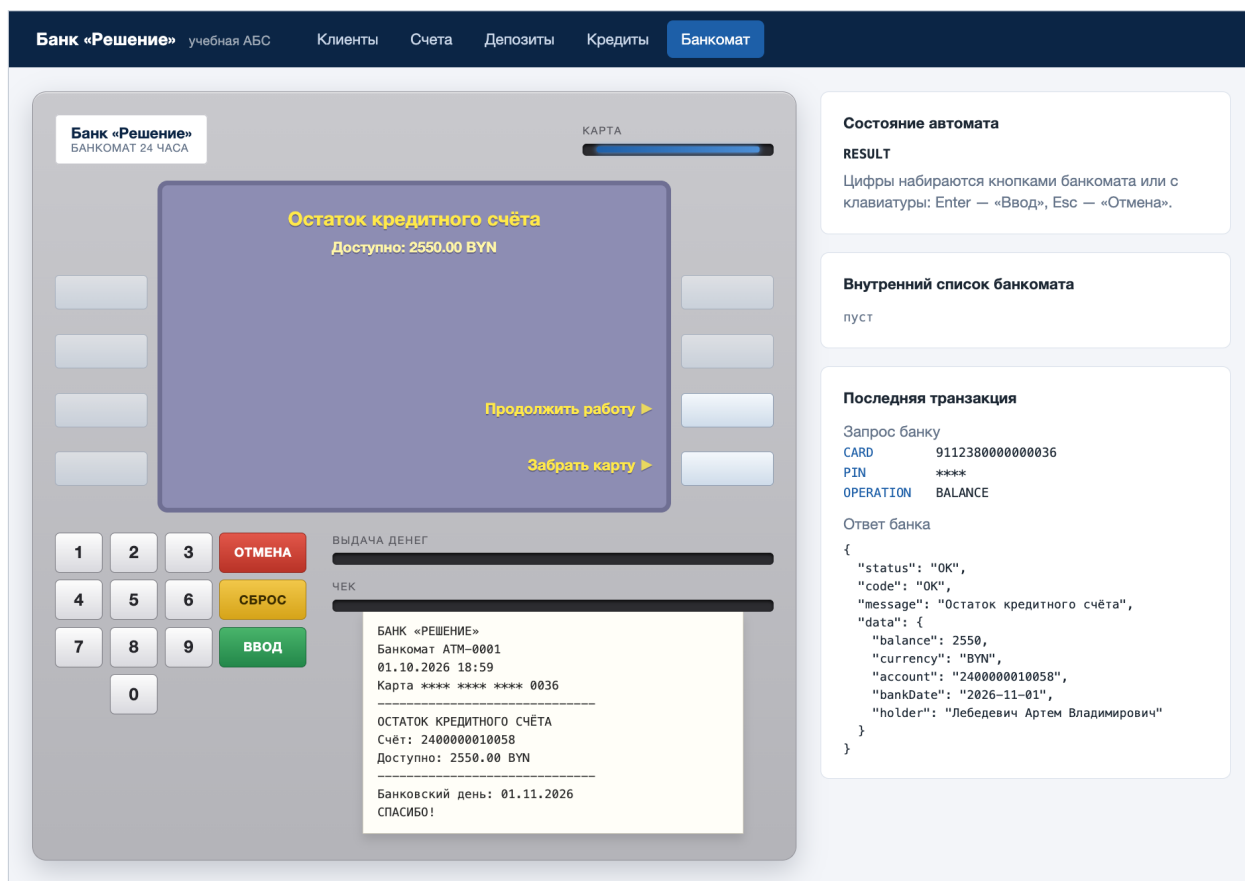


Рисунок 6.8 – Остаток кредитного счёта

Оплата мобильной связи: выбор оператора, ввод номера и суммы, экран подтверждения (рисунок 6.9) и автоматически напечатанный чек (рисунок 6.10).

Банк «Решение» учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Банк «Решение»
БАНКОМАТ 24 ЧАСА

KAPTA

Проверьте данные платежа
Оператор: МТС
Телефон: 0297654321
Сумма: 25 BYN

Подтвердить ▶
Ввести заново ▶

123ОТМЕНА

456СБРОС

789ВВОД

0

ВЫДАЧА ДЕНЕГ
ЧЕК

Состояние автомата

CONFIRM

Цифры набираются кнопками банкомата или с клавиатуры: Enter — «Ввод», Esc — «Отмена».

Внутренний список банкомата

CARD

9112380000000036

PIN

OPERATION

PAYMENT

OPERATOR

MTS

PHONE

0297654321

AMOUNT

25

Последняя транзакция

Запрос банку

CARD

9112380000000036

PIN

OPERATION

AUTHORIZE

Ответ банка

{

"status": "OK",

"code": "OK",

"message": "Авторизация выполнена",

"data": {

"holder": "Лебедевич Артем Владимирович"

}

}

Рисунок 6.9 – Подтверждение данных платежа

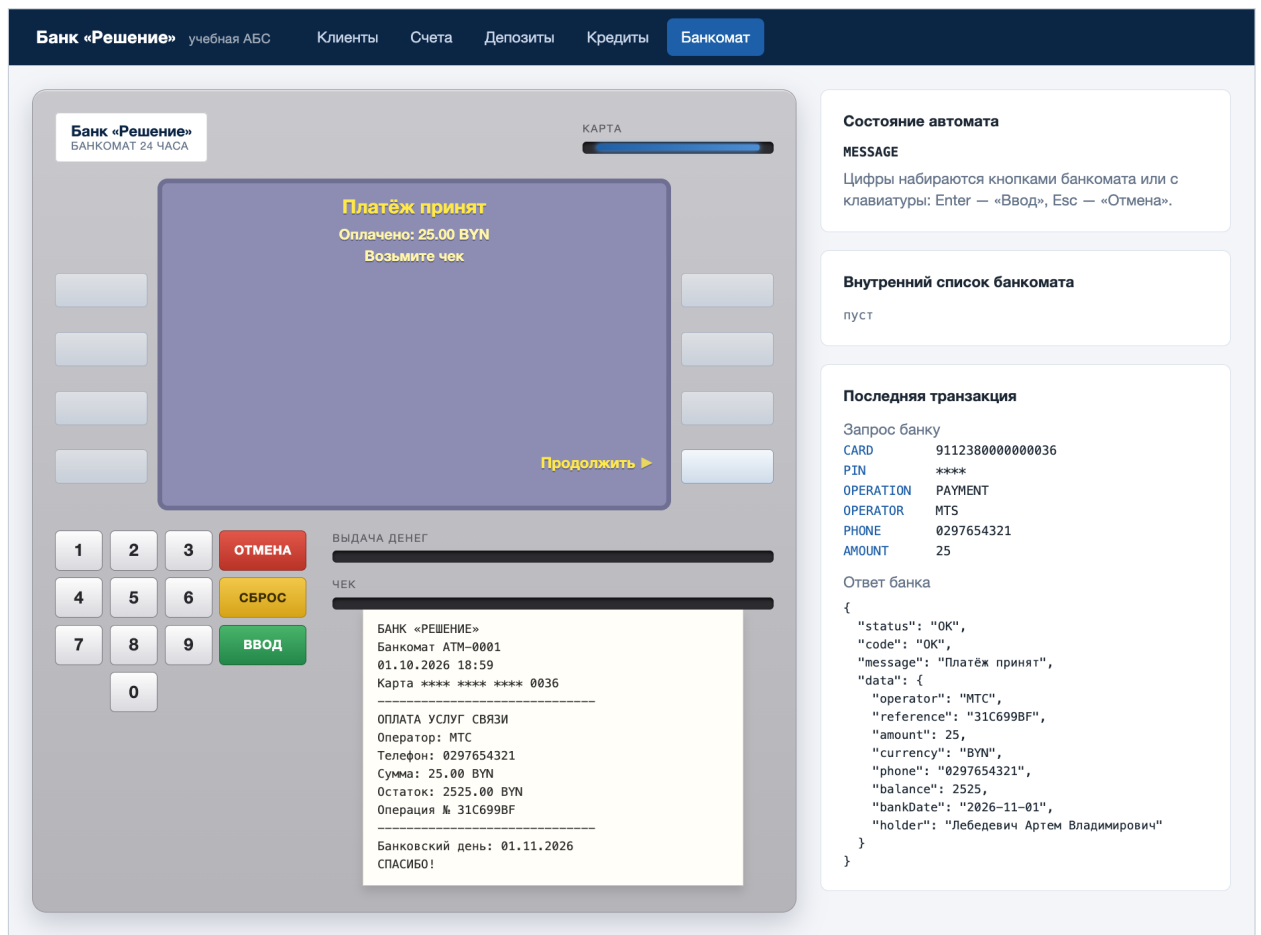


Рисунок 6.10 – Платёж принят, чек напечатан автоматически

При попытке снять больше, чем доступно на счёте, банк отклоняет транзакцию с кодом `INSUFFICIENT_FUNDS`, и банкомат показывает его сообщение (рисунок 6.11). Остаток депозитного счёта держателя карты приведён на рисунке 6.12: данные получены сервисом кредитов от сервиса депозитов.

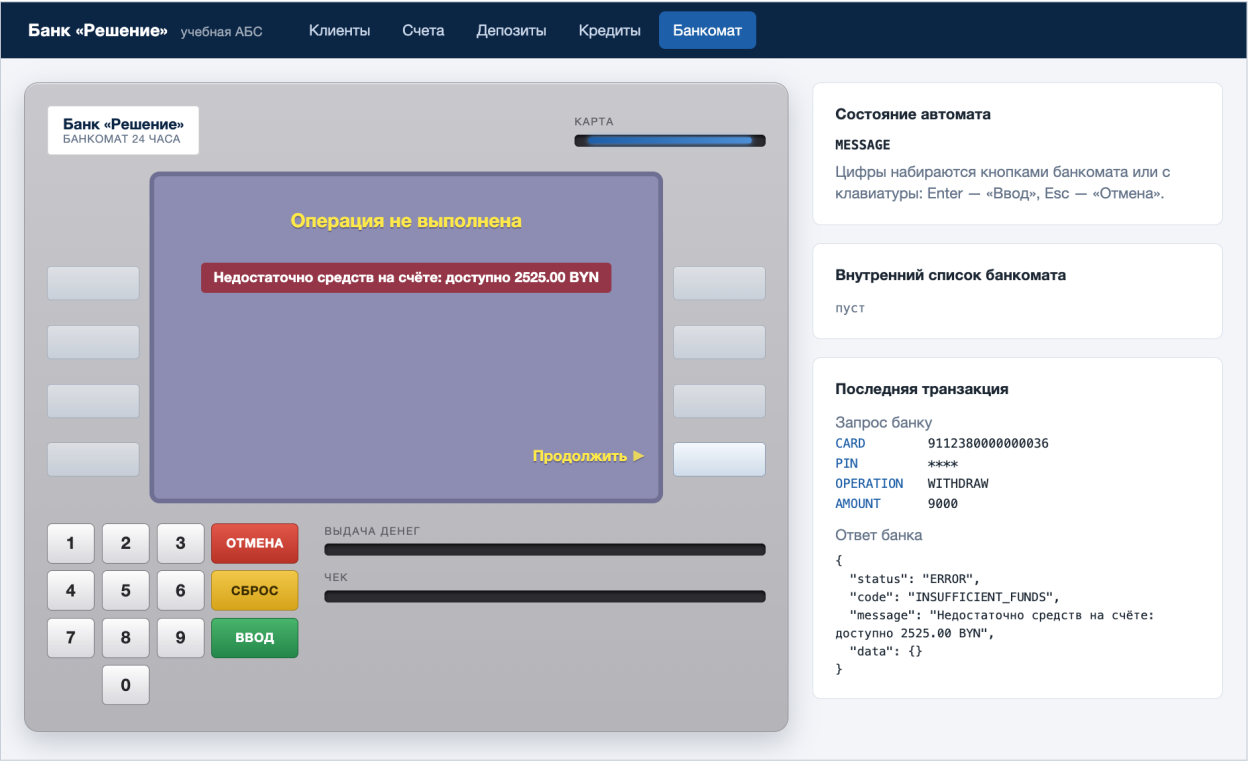


Рисунок 6.11 – Отказ банка: недостаточно средств

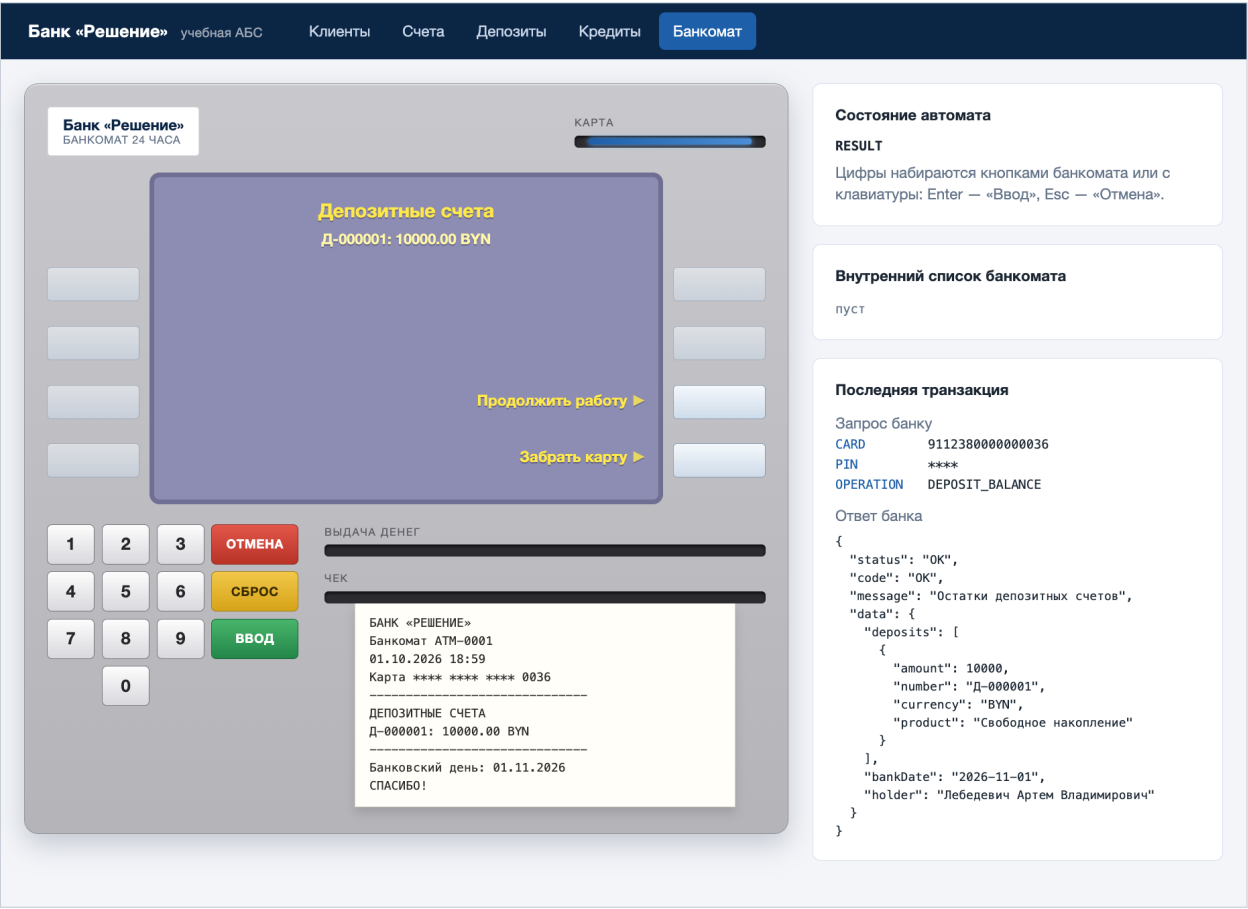


Рисунок 6.12 – Остаток депозитного счёта

После трёх неверных PIN-кодов подряд работа принудительно завер-

шается, а банк блокирует карту (рисунок 6.13).

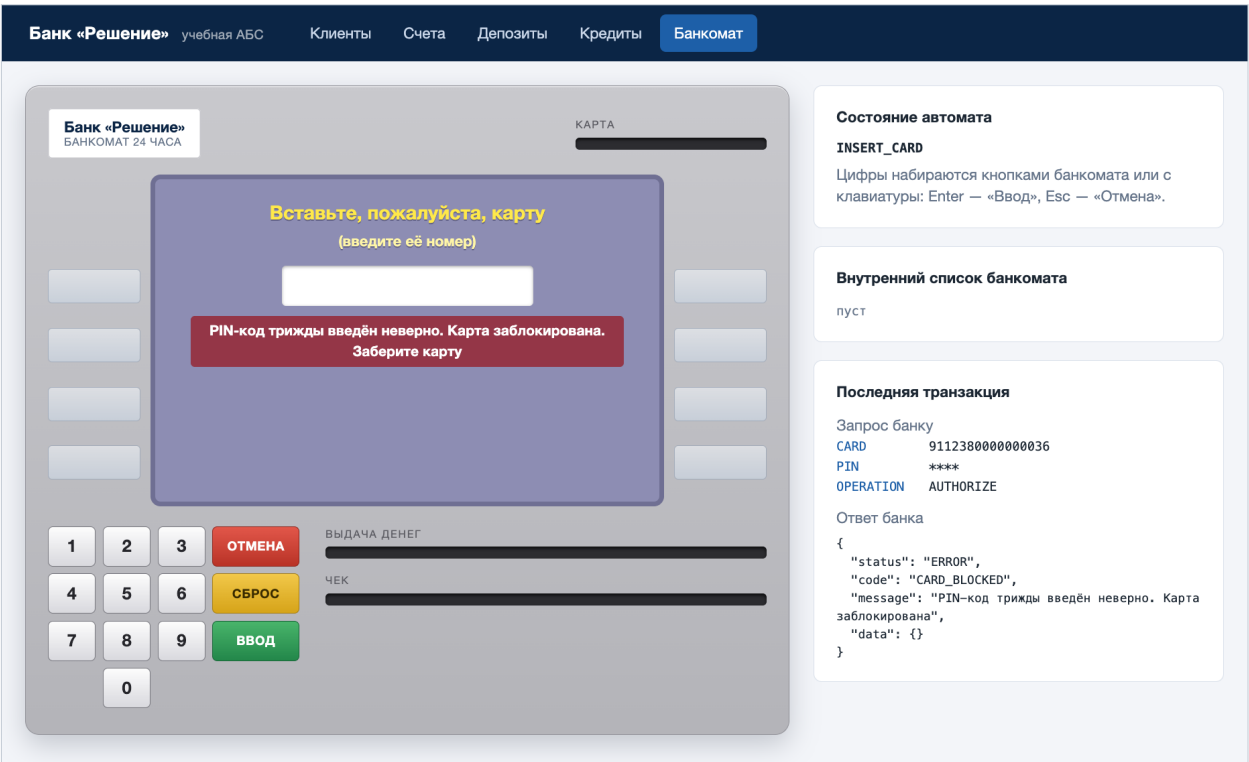


Рисунок 6.13 – Три неверных PIN-кода: карта заблокирована

Результат операций со стороны банка показан на рисунке 6.14: в карточке кредитного договора карта отмечена заблокированной, доступный остаток уменьшился, в проводках видны снятие через кассу и оплата услуг связи. В отчёте главной книги (рисунок 6.15) появился расчётный счёт оператора 3012 с суммой платежа.

Банк «Решение» учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Кредитные договоры

Кредиты физических лиц: заключение договора, график платежей, погашение

БАНКОВСКИЙ ДЕНЬ 01.11.2026

Заккрыть день

30

Заккрыть дней

Договор К-000009 · Лебедевич Артем Владимирович

К списку

ВИД КРЕДИТА

«Р-деньги mix»

Аннуитетные платежи

СУММА КРЕДИТА

3 000,00 BYN

СТАВКА

17.65 % годовых

СТАТУС

Действует

СРОК

12 мес.

ПЕРИОД

01.11.2026 — 01.11.2027

ОСТАТОК ДОЛГА

3 000,00 BYN

УПЛАЧЕНО ПРОЦЕНТОВ

0,00 BYN

Карта и выдача наличных

КАРТА К КРЕДИТНОМУ СЧЁТУ

9112 3800 0000 0036

Заблокирована

ДОСТУПНО НА СЧЁТЕ

2 525,00 BYN

Выдать через кассу, BYN

0.00

Выдать

Перевыпустить PIN

График платежей

№	ДАТА ПЛАТЕЖА	ОСНОВНОЙ ДОЛГ	ПРОЦЕНТЫ	ПЛАТЁЖ	ОСТАТОК ДОЛГА	ОПЛАЧЕН
1	01.12.2026	230,41	44,13	274,54	2 769,59	
2	01.01.2027	233,80	40,74	274,54	2 535,79	
3	01.02.2027	237,24	37,30	274,54	2 298,55	
4	01.03.2027	240,73	33,81	274,54	2 057,82	
5	01.04.2027	244,27	30,27	274,54	1 813,55	
6	01.05.2027	247,87	26,67	274,54	1 565,68	
7	01.06.2027	251,51	23,03	274,54	1 314,17	
8	01.07.2027	255,21	19,33	274,54	1 058,96	
9	01.08.2027	258,96	15,58	274,54	800,00	
10	01.09.2027	262,77	11,77	274,54	537,23	
11	01.10.2027	266,64	7,90	274,54	270,59	
12	01.11.2027	270,59	3,98	274,57	0,00	
Итого, BYN			294,51	3 294,51		

Счета договора

НОМЕР СЧЁТА	КОД	НАЗНАЧЕНИЕ	АКТИВНОСТЬ	ДЕБЕТ	КРЕДИТ	САЛДО
2400000010058	2400	Кредиты физическим лицам Лебедевич Артем Владимирович	Активный	3 000,00	475,00	2 525,00
2470000010060	2470	Начисленные процентные доходы по кредитам физическим лицам Лебедевич Артем Владимирович	Активный	0,00	0,00	0,00

Проводки по договору

ДАТА	ОПЕРАЦИЯ	СЧЁТ	ДЕБЕТ	КРЕДИТ
------	----------	------	-------	--------

Рисунок 6.14 – Карточка кредитного договора после операций в банкомате

Банк «Решение»

учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Счета банка

Отчёт о состоянии счетов и процедура «Закрытие банковского дня»

БАНКОВСКИЙ ДЕНЬ 01.11.2026

Закрыть день

30

Закрыть дней

ОТЧЁТ НА ДАТУ

01.11.2026

СЧЕТОВ В ОТЧЁТЕ

13

ФОНД РАЗВИТИЯ, BYN

99 999 996 078,23

КАССА, BYN

0,00

Отчёт по счетам

Журнал проводок

План счетов

BYN

USD

EUR

НОМЕР СЧЁТА	КОД	НАЗВАНИЕ СЧЁТА	АКТИВНОСТЬ	ДОГОВОР	ДЕБЕТ	КРЕДИТ	САЛДО
1010000000019	1010	Касса банка, BYN Денежные средства в кассе	Активный		29 701,91	29 701,91	0,00
2400000010030	2400	Лебедевич Артем Владимирович Кредиты физическим лицам	Активный	К-000001	13 927,71	13 927,71	0,00
2400000010058	2400	Лебедевич Артем Владимирович Кредиты физическим лицам	Активный	К-000009	3 000,00	475,00	2 525,00
2400000050018	2400	Жуков Дмитрий Александрович Кредиты физическим лицам	Активный	К-000002	6 000,00	0,00	6 000,00
2470000010042	2470	Лебедевич Артем Владимирович Начисленные процентные доходы по кредитам физическим лицам	Активный	К-000001	176,50	176,50	0,00
2470000010060	2470	Лебедевич Артем Владимирович Начисленные процентные доходы по кредитам физическим лицам	Активный	К-000009	0,00	0,00	0,00
2470000050020	2470	Жуков Дмитрий Александрович Начисленные процентные доходы по кредитам физическим лицам	Активный	К-000002	88,25	88,25	0,00
3012000000077	3012	Расчётный счёт оператора МТС Текущие (расчётные) счета коммерческих организаций	Пассивный	OPERATOR-MTS-BYN	0,00	25,00	25,00
3404000010017	3404	Лебедевич Артем Владимирович Вклады (депозиты) до востребования физических лиц	Пассивный	Д-000001	10 000,00	10 000,00	0,00
3414000020015	3414	Савицкая Марина Олеговна Срочные вклады (депозиты) физических лиц	Пассивный	Д-000002	5 000,00	5 000,00	0,00
3470000010025	3470	Лебедевич Артем Владимирович Начисленные процентные расходы по вкладам (депозитам) до востребования	Пассивный	Д-000001	59,45	59,45	0,00
3471000020025	3471	Савицкая Марина Олеговна Начисленные процентные расходы по срочным вкладам (депозитам)	Пассивный	Д-000002	0,00	54,78	54,78
7327000000029	7327	Фонд развития банка, BYN Фонд развития банка	Пассивный		21 114,23	100 000 017 192,46	99 999 996 078,23
Итого обороты, BYN					89 068,05	100 000 076 701,06	

Рисунок 6.15 – Отчёт по счетам: расчётный счёт оператора связи

26

ЗАКЛЮЧЕНИЕ

Разработан эмулятор банкомата, работающий с виртуальным банком из предыдущих работ. Банкомат выполнен отдельным микросервисом: сеанс работы описан конечным автоматом, вводимые данные накапливаются во внутреннем списке, который целиком отправляется банку при совершении операции и после этого очищается. Реализованы все операции задания, включая необязательные: авторизация с ограничением в три попытки, снятие наличных, просмотр остатков кредитного и депозитного счетов, оплата мобильной связи с подтверждением данных, печать чеков и возврат карты.

Банковская сторона протокола — единственная точка входа в сервисе кредитов, которая при каждой транзакции заново проверяет карту и PIN-код и проводит операции через главную книгу. Разделение ответственности получилось таким же, как в реальных системах: устройство ведёт диалог и не имеет доступа к счетам, а банк не хранит состояние диалога и принимает решения по каждой транзакции независимо. PIN-код хранится только в виде хеша, счётчик неверных попыток ведётся на стороне банка.

Решение проверено 68 автоматическими тестами: модульными тестами автомата, интеграционными тестами протокола и функциональными тестами интерфейса через WebDriver.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Хопкрофт, Дж. Введение в теорию автоматов, языков и вычислений / Дж. Хопкрофт, Р. Мотвани, Дж. Ульман. — 2-е изд. — М.: Вильямс, 2008. — 528 с.
2. Ньюмен, С. Создание микросервисов / С. Ньюмен. — 2-е изд. — СПб.: Питер, 2023. — 624 с.
3. ISO/IEC 7812-1:2017. Identification cards — Identification of issuers — Part 1: Numbering system. — Geneva: ISO, 2017. — 9 p.
4. Provos, N. A Future-Adaptable Password Scheme / N. Provos, D. Mazières // Proceedings of the USENIX Annual Technical Conference. — 1999. — P. 81–91.
5. Spring Framework Reference: REST Clients, HTTP Interface [Электронный ресурс]. — Режим доступа: <https://docs.spring.io/spring-framework/reference/integration/rest-clients.html>. — Дата доступа: 01.10.2026.

ПРИЛОЖЕНИЕ А (обязательное)

Исходный код эмулятора банкомата и протокола

Эмулятор банкомата (atm-service)

Листинг А.1 – atm-service/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>by.bsuir.bank</groupId>
    <artifactId>bank</artifactId>
    <version>1.0.0</version>
  </parent>

  <artifactId>atm-service</artifactId>
  <name>atm-service</name>
  <description>JP4. Эмулятор банкомата: конечный автомат сеанса, протокол обмена с банком, web-интерфейс</description>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
      <groupId>by.bsuir.bank</groupId>
      <artifactId>bank-common</artifactId>
    </dependency>
    <dependency>
      <groupId>by.bsuir.bank</groupId>
      <artifactId>ui-kit</artifactId>
    </dependency>
    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <optional>true</optional>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.seleniumhq.selenium</groupId>
      <artifactId>selenium-java</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-failsafe-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

Листинг А.2 – atm-service/src/main/resources/application.yml

```
server:
  port: ${ATM_SERVICE_PORT:8085}

spring:
  application:
    name: atm-service
  http:
    client:
      connect-timeout: 2s
      read-timeout: 15s

atm:
  # Идентификатор устройства печатается на чеках
  terminal-id: ${ATM_TERMINAL_ID:ATM-0001}
  # Процессинг банка – сервис кредитов, к счёту которого привязана карта
  bank-url: ${CREDIT_SERVICE_URL:http://localhost:8084}
```

Листинг А.3 – atm-service/src/main/java/by/bsuir/bank/atm/bank/BankConfig.java

```
package by.bsuir.bank.atm.bank;

import by.bsuir.bank.atm.AtmProperties;
import by.bsuir.bank.common.HttpApis;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.client.RestClient;

import java.time.Clock;

@Configuration
public class BankConfig {

    @Bean
    public BankGateway bankGateway(RestClient.Builder builder, AtmProperties properties) {
        return HttpApis.proxy(builder, properties.bankUrl(), BankGateway.class);
    }

    @Bean
    public Clock clock() {
        return Clock.systemDefaultZone();
    }
}
```

Листинг А.4 – atm-service/src/main/java/by/bsuir/bank/atm/bank/BankGateway.java

```
package by.bsuir.bank.atm.bank;

import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.service.annotation.GetExchange;
import org.springframework.web.service.annotation.HttpExchange;
import org.springframework.web.service.annotation.PostExchange;

import java.util.List;

/** Протокол общения банкомата и банка: банкомат знает о банке только эти два запроса. */
@HttpExchange("/api/atm")
public interface BankGateway {

    @PostExchange("/transactions")
    BankReply send(@RequestBody Transaction transaction);

    @GetExchange("/operators")
    List<Operator> operators();
}
```

Листинг А.5 – atm-service/src/main/java/by/bsuir/bank/atm/bank/BankReply.java

```
package by.bsuir.bank.atm.bank;

import java.util.Map;
```

```

/** Ответ банка на транзакцию: статус OK/ERROR, код, текст для экрана и данные операции. */
public record BankReply(String status, String code, String message, Map<String, Object> data) {

    public boolean ok() {
        return "OK".equals(status);
    }

    public Object get(String key) {
        return data == null ? null : data.get(key);
    }
}

```

Листинг А.6 – atm-service/src/main/java/by/bsuir/bank/atm/bank/Item.java

```

package by.bsuir.bank.atm.bank;

/** Элемент внутреннего списка банкомата: имя и значение. */
public record Item(String name, String value) {

    public static final String CARD = "CARD";
    public static final String PIN = "PIN";
    public static final String OPERATION = "OPERATION";
    public static final String AMOUNT = "AMOUNT";
    public static final String OPERATOR = "OPERATOR";
    public static final String PHONE = "PHONE";

    /** Для показа на экране и в журнале обмена PIN-код заменяется звёздочками. */
    public Item masked() {
        return PIN.equals(name) ? new Item(name, "****") : this;
    }
}

```

Листинг А.7 – atm-service/src/main/java/by/bsuir/bank/atm/bank/Operator.java

```

package by.bsuir.bank.atm.bank;

public record Operator(String code, String name) {
}

```

Листинг А.8 –

atm-service/src/main/java/by/bsuir/bank/atm/bank/Transaction.java

```

package by.bsuir.bank.atm.bank;

import java.util.List;

/** Транзакция – всё содержимое внутреннего списка банкомата, отправляемое банку одним сообщением. */
public record Transaction(List<Item> items) {
}

```

Листинг А.9 –

atm-service/src/main/java/by/bsuir/bank/atm/session/AtmService.java

```

package by.bsuir.bank.atm.session;

import by.bsuir.bank.atm.AtmProperties;
import by.bsuir.bank.atm.bank.BankGateway;
import by.bsuir.bank.common.api.BankException;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.util.Map;
import java.util.UUID;
import java.util.concurrent.ConcurrentHashMap;

/** Сеансы работы с банкоматом: каждый открытый web-интерфейс – отдельный сеанс со своим автоматом. */
@Service
@RequiredArgsConstructor
public class AtmService {

    private static final Duration IDLE_LIMIT = Duration.ofMinutes(15);
}

```

```

private final BankGateway bank;
private final AtmProperties properties;
private final Clock clock;
private final Map<String, AtmSession> sessions = new ConcurrentHashMap<>();

public record SessionView(String id, Screen screen) {
}

public SessionView open() {
    Instant deadline = clock.instant().minus(IDLE_LIMIT);
    sessions.values().removeIf(session -> session.touched().isBefore(deadline));

    String id = UUID.randomUUID().toString();
    AtmSession session = new AtmSession(bank, properties.terminalId(), clock);
    sessions.put(id, session);
    return new SessionView(id, session.screen());
}

public AtmSession get(String id) {
    AtmSession session = sessions.get(id);
    if (session == null) {
        throw BankException.notFound("Сеанс банкомата не найден или истёк");
    }
    return session;
}
}

```

Листинг А.10 –

atm-service/src/main/java/by/bsuir/bank/atm/session/AtmSession.java

```

package by.bsuir.bank.atm.session;

import by.bsuir.bank.atm.bank.BankGateway;
import by.bsuir.bank.atm.bank.BankReply;
import by.bsuir.bank.atm.bank.Item;
import by.bsuir.bank.atm.bank.Operator;
import by.bsuir.bank.atm.bank.Transaction;
import by.bsuir.bank.atm.session.Screen.Exchange;
import by.bsuir.bank.atm.session.Screen.Input;
import by.bsuir.bank.atm.session.Screen.Option;
import org.springframework.web.client.RestClientException;

import java.time.Clock;
import java.time.Instant;
import java.util.ArrayList;
import java.util.List;
import java.util.Set;

/**
 * Сеанс работы с банкоматом – конечный автомат. Вводимые данные накапливаются во внутреннем
 * списке {@link #buffer}; когда для операции собрано всё необходимое, список целиком
 * отправляется банку одной транзакцией и очищается.
 */
public class AtmSession {

    public static final int MAX_PIN_ATTEMPTS = 3;

    private static final String CURRENCY = "BYN";
    private static final Set<String> OPERATIONS = Set.of("WITHDRAW", "BALANCE", "DEPOSIT_BALANCE", "PAYMENT");

    private final BankGateway bank;
    private final String terminalId;
    private final Clock clock;

    private AtmState state = AtmState.INSERT_CARD;
    /** Внутренний список банкомата. */
    private final List<Item> buffer = new ArrayList<>();
    /** Номер «вставленной» карты: устройство помнит его, пока карта не возвращена. */
    private String card;
    private int pinAttempts;
    /** Операция, выбранная в меню до повторного ввода PIN-кода. */
    private String pendingOperation;
    private String operation;
    private BankReply reply;
    private List<Operator> operators = List.of();
    private String notice;
    private String cash;
    private List<String> receipt;
    private Exchange exchange;

```

```

private Instant touched;

public AtmSession(BankGateway bank, String terminalId, Clock clock) {
    this.bank = bank;
    this.terminalId = terminalId;
    this.clock = clock;
    this.touched = clock.instant();
}

public synchronized Instant touched() {
    return touched;
}

// ----- События от клиента -----

/** Клавиша «Ввод»: клиент подтвердил набранное на клавиатуре значение. */
public synchronized Screen enter(String text) {
    begin();
    String value = text == null ? "" : text.replace(" ", "");
    switch (state) {
        case INSERT_CARD -> insertCard(value);
        case PIN -> {
            if (!value.matches("\\d{4}")) {
                notice = "PIN-код состоит из 4 цифр";
            } else {
                buffer.add(new Item(Item.PIN, value));
                authorize();
            }
        }
        case AMOUNT -> {
            if (acceptAmount(value)) {
                execute();
            }
        }
        case PHONE -> {
            if (!value.matches("\\d{10}")) {
                notice = "Номер телефона – 10 цифр, например 0291234567";
            } else {
                buffer.add(new Item(Item.PHONE, value));
                state = AtmState.PAY_AMOUNT;
            }
        }
        case PAY_AMOUNT -> {
            if (acceptAmount(value)) {
                state = AtmState.CONFIRM;
            }
        }
        default -> {
            // в остальных состояниях клавиша «Ввод» ничего не делает
        }
    }
    return screen();
}

/** Боковая кнопка: выбран пункт меню. */
public synchronized Screen select(String option) {
    begin();
    switch (state) {
        case MENU -> {
            if ("EJECT".equals(option)) {
                eject("Заберите карту. Спасибо, что воспользовались нашим банкоматом");
            } else if (OPERATIONS.contains(option)) {
                chooseOperation(option);
            }
        }
        case OPERATOR -> {
            if (operators.stream().anyMatch(o -> o.code().equals(option))) {
                buffer.add(new Item(Item.OPERATOR, option));
                state = AtmState.PHONE;
            }
        }
        case CONFIRM -> {
            if ("CONFIRM".equals(option)) {
                execute();
            } else if ("RETRY".equals(option)) {
                dropFromBuffer(Item.OPERATOR, Item.PHONE, Item.AMOUNT);
                state = AtmState.OPERATOR;
            }
        }
        case RECEIPT_PROMPT -> {
            if ("YES".equals(option)) {
                receipt = Receipts.print(operation, reply, card, terminalId, clock);
            }
        }
    }
}

```

```

    }
    state = "WITHDRAW".equals(operation) ? AtmState.MENU : AtmState.RESULT;
}
case RESULT, MESSAGE -> {
    if ("EJECT".equals(option)) {
        eject("Заберите карту. Спасибо, что воспользовались нашим банкоматом");
    } else {
        state = AtmState.MENU;
    }
}
default -> {
    // в состояниях ввода боковые кнопки не используются
}
}
return screen();
}

/** Клавиша «Отмена»: до входа в меню возвращает карту, внутри операции – возвращает в меню. */
public synchronized Screen cancel() {
    begin();
    switch (state) {
        case INSERT_CARD -> {
        }
        case PIN, MENU -> eject("Операция отменена. Заберите карту");
        default -> {
            dropFromBuffer(Item.OPERATION, Item.AMOUNT, Item.OPERATOR, Item.PHONE);
            state = AtmState.MENU;
            notice = "Операция отменена";
        }
    }
    return screen();
}

// ----- переходы автомата -----

private void begin() {
    touched = clock.instant();
    notice = null;
    cash = null;
    receipt = null;
}

private void insertCard(String number) {
    if (!number.matches("\\d{16}")) {
        notice = "Номер карты состоит из 16 цифр";
    } else if (!luhnValid(number)) {
        notice = "Карта не читается: проверьте номер";
    } else {
        card = number;
        pinAttempts = 0;
        pendingOperation = null;
        buffer.clear();
        buffer.add(new Item(Item.CARD, number));
        state = AtmState.PIN;
    }
}

/** Авторизация карты на сервере банка: во внутреннем списке к этому моменту номер карты и PIN-код. */
private void authorize() {
    List<Item> request = new ArrayList<>(buffer);
    request.add(new Item(Item.OPERATION, "AUTHORIZE"));
    BankReply answer = send(request);
    if (answer == null) {
        dropFromBuffer(Item.PIN);
        notice = "Нет связи с банком. Повторите попытку позже";
    } else if (answer.ok()) {
        pinAttempts = 0;
        if (pendingOperation != null) {
            startOperation(pendingOperation);
        } else {
            state = AtmState.MENU;
        }
    } else if ("WRONG_PIN".equals(answer.code())) {
        dropFromBuffer(Item.PIN);
        pinAttempts++;
        if (pinAttempts >= MAX_PIN_ATTEMPTS) {
            eject("PIN-код введён неверно три раза. Работа завершена, заберите карту");
        } else {
            notice = "Неверный PIN-код. Осталось попыток: " + (MAX_PIN_ATTEMPTS - pinAttempts);
        }
    } else {
        eject(answer.message() + ". Заберите карту");
    }
}

```

```

    }
}

/**
 * Выбор операции в меню. Если предыдущая транзакция уже отправлена и список очищен,
 * данные карты вводятся заново: номер банкомат подставляет сам, PIN-код запрашивает у клиента.
 */
private void chooseOperation(String option) {
    if (hasCredentials()) {
        startOperation(option);
    } else {
        buffer.clear();
        buffer.add(new Item(Item.CARD, card));
        pendingOperation = option;
        state = AtmState.PIN;
    }
}

private void startOperation(String option) {
    operation = option;
    pendingOperation = null;
    buffer.add(new Item(Item.OPERATION, option));
    switch (option) {
        case "WITHDRAW" -> state = AtmState.AMOUNT;
        case "PAYMENT" -> {
            try {
                operators = bank.operators();
                state = AtmState.OPERATOR;
            } catch (RestClientException e) {
                dropFromBuffer(Item.OPERATION);
                state = AtmState.MENU;
                notice = "Нет связи с банком. Платежи временно недоступны";
            }
        }
        default -> execute(); // BALANCE, DEPOSIT_BALANCE: дополнительных данных не требуется
    }
}

private boolean acceptAmount(String value) {
    if (!value.matches("\\d{1,7}")) {
        notice = "Сумма – целое неотрицательное число";
        return false;
    }
    if (Integer.parseInt(value) == 0) {
        notice = "Сумма должна быть больше нуля";
        return false;
    }
    buffer.add(new Item(Item.AMOUNT, String.valueOf(Integer.parseInt(value))));
    return true;
}

/** Отправка накопленной транзакции: содержимое списка уходит банку целиком, список очищается. */
private void execute() {
    reply = send(new ArrayList<>(buffer));
    buffer.clear();
    if (reply == null) {
        state = AtmState.MESSAGE;
        notice = "Нет связи с банком. Операция не выполнена";
    } else if (!reply.ok()) {
        if ("WRONG_PIN".equals(reply.code()) || "CARD_BLOCKED".equals(reply.code())) {
            eject(reply.message() + ". Заберите карту");
        } else {
            state = AtmState.MESSAGE;
            notice = reply.message();
        }
    } else if ("PAYMENT".equals(operation)) {
        receipt = Receipts.print(operation, reply, card, terminalId, clock); // чек платежа печатается всегда
        state = AtmState.MESSAGE;
    } else {
        if ("WITHDRAW".equals(operation)) {
            cash = Receipts.money(reply, "amount");
        }
        state = AtmState.RECEIPT_PROMPT;
    }
}

private BankReply send(List<Item> items) {
    List<Item> masked = items.stream().map(Item::masked).toList();
    try {
        BankReply answer = bank.send(new Transaction(List.copyOf(items)));
        exchange = new Exchange(masked, answer);
        return answer;
    }
}

```

```

    } catch (RestClientException e) {
        exchange = new Exchange(masked, null);
        return null;
    }
}

/** Возврат карты: сеанс завершается, банкомат снова ждёт карту. */
private void eject(String message) {
    buffer.clear();
    card = null;
    pinAttempts = 0;
    pendingOperation = null;
    operation = null;
    reply = null;
    state = AtmState.INSERT_CARD;
    notice = message;
}

private boolean hasCredentials() {
    return has(Item.CARD) && has(Item.PIN);
}

private boolean has(String name) {
    return buffer.stream().anyMatch(item -> item.name().equals(name));
}

private void dropFromBuffer(String... names) {
    Set<String> drop = Set.of(names);
    buffer.removeIf(item -> drop.contains(item.name()));
}

private String valueOf(String name) {
    return buffer.stream().filter(item -> item.name().equals(name)).map(Item::value).findFirst().orElse("");
}

/** Контрольная цифра номера карты по алгоритму Луна — так банкомат отсеивает опечатки без запроса в банк. */
static boolean luhnValid(String number) {
    int sum = 0;
    boolean doubled = false;
    for (int i = number.length() - 1; i >= 0; i--) {
        int digit = number.charAt(i) - '0';
        if (doubled) {
            digit *= 2;
            if (digit > 9) {
                digit -= 9;
            }
        }
        sum += digit;
        doubled = !doubled;
    }
    return sum % 10 == 0;
}

// ----- экран -----

public synchronized Screen screen() {
    String title;
    List<String> lines = List.of();
    Input input = null;
    List<Option> options = List.of();
    switch (state) {
        case INSERT_CARD -> {
            title = "Вставьте, пожалуйста, карту";
            lines = List.of("(введите её номер)");
            input = new Input("CARD", 16, true);
        }
        case PIN -> {
            title = pendingOperation == null ? "Введите PIN-код" : "Введите PIN-код ещё раз";
            lines = pendingOperation == null ? List.of() : List.of("Для новой операции данные карты вводятся заново");
            input = new Input("PIN", 4, true);
        }
        case MENU -> {
            title = "Выберите операцию";
            options = List.of(
                new Option("WITHDRAW", "Снять наличные"),
                new Option("BALANCE", "Остаток кредитного счёта"),
                new Option("DEPOSIT_BALANCE", "Остаток депозитного счёта"),
                new Option("PAYMENT", "Оплата мобильной связи"),
                new Option("EJECT", "Забрать карту");
            );
        }
        case AMOUNT -> {
            title = "Введите сумму";
        }
    }
}

```

```

        lines = List.of("Снятие наличных с кредитного счёта, " + CURRENCY);
        input = new Input("NUMBER", 7, false);
    }
    case OPERATOR -> {
        title = "Выберите оператора связи";
        options = operators.stream().map(o -> new Option(o.code(), o.name())).toList();
    }
    case PHONE -> {
        title = "Введите номер телефона";
        lines = List.of("10 цифр, например 0291234567");
        input = new Input("NUMBER", 10, true);
    }
    case PAY_AMOUNT -> {
        title = "Введите сумму платежа";
        lines = List.of("Оплата мобильной связи, " + CURRENCY);
        input = new Input("NUMBER", 7, false);
    }
    case CONFIRM -> {
        title = "Проверьте данные платежа";
        String code = valueOf(Item.OPERATOR);
        String name = operators.stream().filter(o -> o.code().equals(code)).map(Operator::name).findFirst().orElse(code);
        lines = List.of("Оператор: " + name, "Телефон: " + valueOf(Item.PHONE),
            "Сумма: " + valueOf(Item.AMOUNT) + " " + CURRENCY);
        options = List.of(new Option("CONFIRM", "Подтвердить"), new Option("RETRY", "Ввести заново"));
    }
    case RECEIPT_PROMPT -> {
        title = "WITHDRAW.equals(operation) ? Заберите деньги" : "Данные получены";
        lines = "WITHDRAW.equals(operation)
            ? List.of("Выдано: " + Receipts.money(reply, "amount"), "Распечатать чек?")
            : List.of("Распечатать чек?");
        options = List.of(new Option("YES", "Да"), new Option("NO", "Нет"));
    }
    case RESULT -> {
        if ("BALANCE".equals(operation)) {
            title = "Остаток кредитного счёта";
            lines = List.of("Доступно: " + Receipts.money(reply, "balance"));
        } else {
            title = "Депозитные счета";
            lines = Receipts.deposits(reply);
        }
        options = List.of(new Option("CONTINUE", "Продолжить работу"), new Option("EJECT", "Забрать карту"));
    }
    default -> {
        boolean success = reply != null && reply.ok();
        title = success ? "Платёж принят" : "Операция не выполнена";
        lines = success ? List.of("Оплачено: " + Receipts.money(reply, "amount"), "Возьмите чек") : List.of();
        options = List.of(new Option("CONTINUE", "Продолжить"));
    }
}
return new Screen(state.name(), title, lines, input, options, notice, card != null, cash, receipt,
    buffer.stream().map(Item::masked).toList(), exchange);
}
}

```

Листинг А.11 –

atm-service/src/main/java/by/bsuir/bank/atm/session/AtmState.java

```

package by.bsuir.bank.atm.session;

/** Состояния конечного автомата сеанса работы с банкоматом. */
public enum AtmState {
    /** Ожидание карты: ввод её номера. */
    INSERT_CARD,
    /** Ввод PIN-кода – при входе и перед каждой следующей операцией. */
    PIN,
    /** Главное меню. */
    MENU,
    /** Снятие наличных: ввод суммы. */
    AMOUNT,
    /** Платёж: выбор оператора связи. */
    OPERATOR,
    /** Платёж: ввод номера телефона. */
    PHONE,
    /** Платёж: ввод суммы. */
    PAY_AMOUNT,
    /** Платёж: подтверждение введённых данных. */
    CONFIRM,
    /** Операция выполнена: вопрос о печати чека. */
    RECEIPT_PROMPT,
    /** Показ состояния счёта: продолжить работу или забрать карту. */
}

```

```

    RESULT,
    /** Сообщение о результате операции или об ошибке. */
    MESSAGE
}

```

Листинг А.12 –

atm-service/src/main/java/by/bsuir/bank/atm/session/Receipts.java

```

package by.bsuir.bank.atm.session;

import by.bsuir.bank.atm.bank.BankReply;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.Clock;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;

/** Печать чеков: текстовые строки по данным ответа банка. */
public final class Receipts {

    private static final DateTimeFormatter TIME = DateTimeFormatter.ofPattern("dd.MM.yyyy HH:mm");
    private static final DateTimeFormatter DAY = DateTimeFormatter.ofPattern("dd.MM.yyyy");
    private static final String RULE = "-----";

    public static List<String> print(String operation, BankReply reply, String card, String terminalId, Clock clock) {
        List<String> lines = new ArrayList<>();
        lines.add("БАНК «РЕШЕНИЕ»");
        lines.add("Банкомат " + terminalId);
        lines.add(TIME.format(LocalDateTime.now(clock)));
        lines.add("Карта **** * " + card.substring(card.length() - 4));
        lines.add(RULE);
        switch (operation) {
            case "WITHDRAW" -> {
                lines.add("ВЫДАЧА НАЛИЧНЫХ");
                lines.add("Сумма: " + money(reply, "amount"));
                lines.add("Остаток: " + money(reply, "balance"));
            }
            case "BALANCE" -> {
                lines.add("ОСТАТОК КРЕДИТНОГО СЧЁТА");
                lines.add("Счёт: " + reply.get("account"));
                lines.add("Доступно: " + money(reply, "balance"));
            }
            case "DEPOSIT_BALANCE" -> {
                lines.add("ДЕПОЗИТНЫЕ СЧЕТА");
                lines.addAll(deposits(reply));
            }
            case "PAYMENT" -> {
                lines.add("ОПЛАТА УСЛУГ СВЯЗИ");
                lines.add("Оператор: " + reply.get("operator"));
                lines.add("Телефон: " + reply.get("phone"));
                lines.add("Сумма: " + money(reply, "amount"));
                lines.add("Остаток: " + money(reply, "balance"));
                lines.add("Операция № " + reply.get("reference"));
            }
            default -> lines.add(operation);
        }
        lines.add(RULE);
        lines.add("Банковский день: " + DAY.format(LocalDate.parse(String.valueOf(reply.get("bankDate")))));
        lines.add("СПАСИБО!");
        return lines;
    }

    /** Строки с остатками вкладов — для чека и для экрана. */
    @SuppressWarnings("unchecked")
    public static List<String> deposits(BankReply reply) {
        List<Map<String, Object>> deposits = (List<Map<String, Object>>) reply.get("deposits");
        if (deposits == null || deposits.isEmpty()) {
            return List.of("Действующих вкладов нет");
        }
        return deposits.stream()
            .map(d -> d.get("number") + ": " + format(d.get("amount")) + " " + d.get("currency"))
            .toList();
    }

    public static String money(BankReply reply, String key) {

```

```

        return format(reply.get(key)) + " " + reply.get("currency");
    }

    private static String format(Object value) {
        return new BigDecimal(String.valueOf(value)).setScale(2, RoundingMode.HALF_UP).toPlainString();
    }

    private Receipts() {
    }
}

```

Листинг А.13 – atm-service/src/main/java/by/bsuir/bank/atm/session/Screen.java

```

package by.bsuir.bank.atm.session;

import by.bsuir.bank.atm.bank.BankReply;
import by.bsuir.bank.atm.bank.Item;

import java.util.List;

/**
 * Всё, что видит клиент банкомата: содержимое экрана, ожидаемый ввод, пункты меню у боковых кнопок,
 * выданные деньги и чек. Для учебных целей сюда же выводятся внутренний список и последний обмен с банком.
 */
public record Screen(
    String state,
    String title,
    List<String> lines,
    Input input,
    List<Option> options,
    String notice,
    boolean cardInside,
    String cash,
    List<String> receipt,
    List<Item> buffer,
    Exchange exchange
) {

    /** Ожидаемый ввод с клавиатуры: только цифры, length — их количество (exact) либо максимум. */
    public record Input(String kind, int length, boolean exact) {
    }

    public record Option(String key, String label) {
    }

    /** Последняя транзакция, отправленная банку, и его ответ (PIN-код скрыт). */
    public record Exchange(List<Item> request, BankReply reply) {
    }
}

```

Листинг А.14 – atm-service/src/main/java/by/bsuir/bank/atm/AtmProperties.java

```

package by.bsuir.bank.atm;

import org.springframework.boot.context.properties.ConfigurationProperties;

@ConfigurationProperties("atm")
public record AtmProperties(String terminalId, String bankUrl) {
}

```

Листинг А.15 – atm-service/src/main/java/by/bsuir/bank/atm/AtmServiceApplication.java

```

package by.bsuir.bank.atm;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.context.properties.ConfigurationPropertiesScan;

/** Из общей библиотеки подключается только обработчик ошибок. */
@SpringBootApplication(scanBasePackages = {"by.bsuir.bank.atm", "by.bsuir.bank.common.api"})
@ConfigurationPropertiesScan
public class AtmServiceApplication {
}

```

```

    public static void main(String[] args) {
        SpringApplication.run(AtmServiceApplication.class, args);
    }
}

```

Листинг А.16 –

atm-service/src/main/java/by/bsuir/bank/atm/web/AtmController.java

```

package by.bsuir.bank.atm.web;

import by.bsuir.bank.atm.session.AtmService;
import by.bsuir.bank.atm.session.AtmService.SessionView;
import by.bsuir.bank.atm.session.Screen;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseBody;
import org.springframework.web.bind.annotation.RestController;

/** REST клиентского интерфейса банкомата: события клавиатуры и боковых кнопок, в ответ – новый экран. */
@RestController
@RequestMapping("/api/sessions")
@RequiredArgsConstructor
public class AtmController {

    private final AtmService service;

    public record Entered(String value) {
    }

    public record Selected(String option) {
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public SessionView open() {
        return service.open();
    }

    @GetMapping("/{id}")
    public Screen screen(@PathVariable String id) {
        return service.get(id).screen();
    }

    @PostMapping("/{id}/enter")
    public Screen enter(@PathVariable String id, @RequestBody Entered event) {
        return service.get(id).enter(event.value());
    }

    @PostMapping("/{id}/select")
    public Screen select(@PathVariable String id, @RequestBody Selected event) {
        return service.get(id).select(event.option());
    }

    @PostMapping("/{id}/cancel")
    public Screen cancel(@PathVariable String id) {
        return service.get(id).cancel();
    }
}

```

Листинг А.17 – atm-service/src/main/resources/static/index.html

```

<!doctype html>
<html lang="ru">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Банкомат – Банк «Решение»</title>
    <link rel="stylesheet" href="/ui/bank.css">
    <link rel="stylesheet" href="/atm.css">
</head>
<body>
    <div id="app" v-cloak></div>

```

```

<script src="/webjars/vue/dist/vue.global.prod.js"></script>
<script src="/ui/bank.js"></script>
<script src="/app.js"></script>
</body>
</html>

```

Листинг А.18 – atm-service/src/main/resources/static/app.js

```

/* Клиентский интерфейс банкомата: «тонкий» терминал. Вся логика сеанса — на сервере (AtmSession),
   сюда приходит готовое описание экрана, отсюда уходят события: ввод, выбор пункта меню, отмена. */

const app = Vue.createApp({
  data() {
    return {
      sessionId: null,
      screen: null,
      typed: '',
      busy: false,
      busyText: '',
      error: '',
    };
  },
  computed: {
    /* Пункты меню прижимаются к нижним боковым кнопкам: первые четыре — справа, остальные — слева. */
    rightOptions() {
      return this.column(this.screen.options.slice(0, 4));
    },
    leftOptions() {
      return this.column(this.screen.options.slice(4, 8));
    },
    shownInput() {
      const input = this.screen.input;
      if (!input) return '';
      if (input.kind === 'PIN') return '●'.repeat(this.typed.length);
      if (input.kind === 'CARD') return this.typed.replace(/(\d{4})(?=\d)/g, '$1 ');
      return this.typed;
    },
    reply() {
      const exchange = this.screen.exchange;
      return exchange && exchange.reply ? JSON.stringify(exchange.reply, null, 2) : 'нет ответа';
    },
  },
  async mounted() {
    window.addEventListener('keydown', this.keyboard);
    await this.open();
  },
  methods: {
    column(options) {
      const rows = [null, null, null, null];
      options.forEach((option, i) => { rows[4 - options.length + i] = option; });
      return rows;
    },
    async open() {
      try {
        const session = await Bank.api('POST', '/api/sessions');
        this.sessionId = session.id;
        this.screen = session.screen;
      } catch (e) {
        this.error = e.message;
      }
    },
    /* Отправка события автомату банкомата. Пока банк отвечает, на экране — «Авторизация на сервере...». */
    async send(action, body, waitText) {
      if (this.busy) return;
      this.busy = true;
      this.busyText = waitText;
      this.error = '';
      try {
        const pause = new Promise(resolve => setTimeout(resolve, waitText ? 500 : 0));
        const next = await Bank.api('POST', '/api/sessions/' + this.sessionId + '/' + action, body);
        await pause;
        this.screen = next;
        this.typed = '';
      } catch (e) {
        if (e.status === 404) await this.open(); // сервис перезапущен — начинаем новый сеанс
        else this.error = e.message;
      } finally {
        this.busy = false;
      }
    }
  }
});

```

```

},
digit(d) {
  const input = this.screen.input;
  if (input && !this.busy && this.typed.length < input.length) this.typed += d;
},
clear() {
  this.typed = '';
},
enter() {
  if (!this.screen.input) return;
  const state = this.screen.state;
  const wait = state === 'PIN' ? 'Авторизация на сервере...'
    : (state === 'AMOUNT' ? 'Запрос в банк...' : '');
  this.send('enter', { value: this.typed }, wait);
},
select(option) {
  if (!option) return;
  // запрос остатка уходит в банк сразу, только если в списке ещё лежат данные карты
  const hasPin = this.screen.buffer.some(item => item.name === 'PIN');
  const toBank = option.key === 'CONFIRM' || (hasPin && ['BALANCE', 'DEPOSIT_BALANCE'].includes(option.key));
  this.send('select', { option: option.key }, toBank ? 'Запрос в банк...' : '');
},
cancel() {
  this.send('cancel', undefined, '');
},
keyboard(event) {
  if (!this.screen) return;
  if (/^\d$/.test(event.key)) this.digit(event.key);
  else if (event.key === 'Enter') this.enter();
  else if (event.key === 'Escape') this.cancel();
  else if (event.key === 'Backspace') this.typed = this.typed.slice(0, -1);
},
},
template: `
<bank-nav active="atm"></bank-nav>
<main v-if="screen" class="atm-page">
  <div>
    <div v-if="error" id="error" class="alert error">{{ error }}</div>
    <div class="atm">
      <div class="atm-head">
        <div class="atm-logo">Банк «Решение»<small>БАНКОМАТ 24 ЧАСА</small></div>
        <div class="card-slot">
          <div class="slot-label">Карты</div>
          <div class="slot" :class="{ inserted: screen.cardInside }" id="card-slot"></div>
        </div>
      </div>
      <div class="atm-body">
        <div class="side">
          <button v-for="(option, i) in leftOptions" :key="'l' + i" :id="option ? 'opt-' + option.key : null"
            :disabled="!option || busy" @click="select(option)"></button>
        </div>
        <div class="display" id="display" :data-state="screen.state">
          <div class="display-top">
            <div class="display-title" id="display-title">{{ busy && busyText ? busyText : screen.title }}</div>
            <div v-if="!(busy && busyText)" class="display-lines" id="display-lines">
              <div v-for="(line, i) in screen.lines" :key="i">{{ line }}</div>
            </div>
          </div>
          <div class="display-main">
            <div class="display-center" v-if="!(busy && busyText)">
              <div v-if="screen.input" class="display-input" id="display-input">{{ shownInput }}</div>
              <div v-if="screen.notice" class="display-notice" id="notice"
                :class="{ ok: screen.state === 'INSERT_CARD' && screen.notice.includes('Спасибо') }">{{ screen.notice }}</div>
            </div>
            <div class="display-row" v-for="row in 4" :key="row">
              <span class="opt left" v-if="leftOptions[row - 1] && !busy">{{ leftOptions[row - 1].label }}</span>
              <span class="opt right" v-if="rightOptions[row - 1] && !busy">{{ rightOptions[row - 1].label }}</span>
            </div>
          </div>
          <div class="side">
            <button v-for="(option, i) in rightOptions" :key="'r' + i" :id="option ? 'opt-' + option.key : null"
              :disabled="!option || busy" @click="select(option)"></button>
          </div>
        </div>
      </div>
      <div class="atm-foot">
        <div class="keypad">

```

```

<button v-for="d in ['1', '2', '3'] :key="d" :id="'key-' + d" @click="digit(d)">{{ d }}</button>
<button id="key-cancel" class="fn cancel" @click="cancel">Отмена</button>
<button v-for="d in ['4', '5', '6'] :key="d" :id="'key-' + d" @click="digit(d)">{{ d }}</button>
<button id="key-clear" class="fn clear" @click="clear">Сброс</button>
<button v-for="d in ['7', '8', '9'] :key="d" :id="'key-' + d" @click="digit(d)">{{ d }}</button>
<button id="key-enter" class="fn enter" @click="enter">Ввод</button>
<span></span>
<button id="key-0" @click="digit('0')">0</button>
</div>
<div class="outputs">
  <div>
    <div class="slot-label">Выдача денег</div>
    <div class="slot"></div>
    <div v-if="screen.cash" class="cash-notes" id="cash">{{ screen.cash }}</div>
  </div>
  <div>
    <div class="slot-label">Чек</div>
    <div class="slot"></div>
    <div v-if="screen.receipt" class="receipt" id="receipt">{{ screen.receipt.join('\n') }}</div>
  </div>
</div>
</div>
</div>
<aside class="inspector">
  <div class="card">
    <h2>Состояние автомата</h2>
    <div class="state" id="state">{{ screen.state }}</div>
    <p class="muted" style="margin: 8px 0 0 0">Цифры набираются кнопками банкомата или с клавиатуры: Enter – «Ввод», Esc – «Отмена».</p>
  </div>
  <div class="card">
    <h2>Внутренний список банкомата</h2>
    <div class="mono-list" id="buffer">
      <div v-for="(item, i) in screen.buffer" :key="i"><span class="name">{{ item.name }}</span>{{ item.value }}</div>
      <div v-if="!screen.buffer.length" class="muted">пуст</div>
    </div>
  </div>
  <div class="card" v-if="screen.exchange">
    <h2>Последняя транзакция</h2>
    <div class="muted">Запрос банку</div>
    <div class="mono-list" id="request">
      <div v-for="(item, i) in screen.exchange.request" :key="i"><span class="name">{{ item.name }}</span>{{ item.value }}</div>
    </div>
    <div class="muted" style="margin-top: 10px">Ответ банка</div>
    <pre id="reply">{{ reply }}</pre>
  </div>
</aside>
</main>`,
});

```

```

/* Оформление эмулятора банкомата */
.atm-page { display: grid; grid-template-columns: minmax(640px, 760px) 1fr; gap: 24px; align-items: start; }

.atm {
  background: linear-gradient(#c9c9cd, #b4b4b9);
  border: 1px solid #8e8e95;
  border-radius: 14px;
  padding: 22px 22px 26px;
  box-shadow: 0 10px 28px rgba(20, 30, 50, .22), inset 0 1px 0 rgba(255, 255, 255, .6);
}

.atm-head { display: flex; align-items: center; justify-content: space-between; margin-bottom: 16px; }
.atm-logo {
  background: #fff; border-radius: 4px; padding: 10px 16px; font-weight: 700; color: #0b2545;
  box-shadow: inset 0 0 1px #c6c6cc; line-height: 1.15;
}
.atm-logo small { display: block; font-weight: 400; font-size: 11px; color: #66758a; letter-spacing: .5px; }
.atm-terminal { font: 12px "SF Mono", Menlo, Consolas, monospace; color: #55555c; }

.slot { position: relative; background: #2c2c30; border-radius: 4px; height: 12px; box-shadow: inset 0 2px 3px rgba(0, 0, 0, .7);
}
.slot-label { font-size: 10.5px; letter-spacing: 1.2px; text-transform: uppercase; color: #5d5d64; margin-bottom: 5px; }

```

```

.card-slot { width: 190px; }
.card-slot .slot.inserted::after {
  content: ""; position: absolute; left: 14px; right: 14px; top: 3px; height: 6px; border-radius: 2px;
  background: linear-gradient(90deg, #1d5fa8, #4d8fd6); box-shadow: 0 0 6px #6fb1ff;
}

.atm-body { display: grid; grid-template-columns: 92px 1fr 92px; gap: 10px; align-items: stretch; }
.side { display: grid; grid-template-rows: repeat(4, 1fr); gap: 0; padding: 82px 0 14px; }
.side button {
  align-self: center; height: 34px; border: 1px solid #8c9db3; border-radius: 4px;
  background: linear-gradient(#f4f8fc, #cfdcea); cursor: pointer; padding: 0;
}
.side button:hover:not(:disabled) { background: linear-gradient(#fff, #dbe7f3); border-color: #1d5fa8; }
.side button:disabled { opacity: .55; cursor: default; }

.display {
  background: #8d8db3; border: 5px solid #6f6f93; border-radius: 10px; min-height: 330px;
  color: #ffe94a; font-weight: 700; text-shadow: 0 1px 0 rgba(40, 40, 80, .45);
  display: grid; grid-template-rows: 82px 1fr; position: relative; overflow: hidden;
}
.display-top { display: flex; flex-direction: column; align-items: center; justify-content: center; text-align: center; padding:
  10px 18px 0; }
.display-title { font-size: 19px; }
.display-lines { font-size: 14.5px; font-weight: 600; margin-top: 4px; color: #fff6a8; }
.display-main { display: grid; grid-template-rows: repeat(4, 1fr); padding: 0 12px 14px; position: relative; }
.display-row { display: flex; align-items: center; justify-content: space-between; gap: 12px; font-size: 15px; }
.display-row .opt { max-width: 48%; }
.display-row .opt.right { text-align: right; margin-left: auto; }
.display-row .opt.left::before { content: "◀ "; opacity: .8; }
.display-row .opt.right::after { content: "▶ "; opacity: .8; }
.display-center { position: absolute; left: 0; right: 0; top: 4px; display: flex; flex-direction: column; align-items: center; gap
  : 10px; pointer-events: none; }
.display-input {
  min-width: 250px; min-height: 40px; padding: 6px 14px; background: #fff; color: #1c2733; border-radius: 4px;
  font: 600 21px "SF Mono", Menlo, Consolas, monospace; letter-spacing: 2px; text-align: center; text-shadow: none;
  box-shadow: inset 0 1px 3px rgba(0, 0, 0, .35);
}
.display-notice { color: #fff; background: rgba(150, 30, 40, .78); padding: 5px 12px; border-radius: 4px; font-size: 14px; font-
  weight: 600; text-shadow: none; max-width: 86%; text-align: center; }
.display-notice.ok { background: rgba(25, 110, 60, .75); }

.atm-foot { display: grid; grid-template-columns: 250px 1fr; gap: 26px; margin-top: 20px; align-items: start; }
.keypad { display: grid; grid-template-columns: repeat(3, 1fr) 1.25fr; gap: 7px; }
.keypad button {
  height: 40px; font-size: 17px; font-weight: 600; border-radius: 5px; border: 1px solid #77777e;
  background: linear-gradient(#dfdffd, #d9d9de); color: #26262b; cursor: pointer;
}
.keypad button:active { transform: translateY(1px); }
.keypad .fn { font-size: 12.5px; color: #fff; text-transform: uppercase; letter-spacing: .4px; }
.keypad .cancel { background: linear-gradient(#e2574a, #b93327); border-color: #8e241b; }
.keypad .clear { background: linear-gradient(#f2c84a, #d6a419); border-color: #a67d0e; color: #3a2c00; }
.keypad .enter { background: linear-gradient(#49b56b, #23874a); border-color: #176537; }

.outputs { display: grid; gap: 14px; }
.cash-notes {
  margin: 0 18px; background: linear-gradient(#e9f6ec, #c9e8d1); border: 1px solid #7fb893; border-top: none;
  border-radius: 0 0 4px 4px; padding: 9px 10px; text-align: center; color: #17603a; font-weight: 700;
}
.receipt {
  margin: 0 30px; background: #fffef8; border: 1px solid #d9d6c5; border-top: none; padding: 10px 14px 12px;
  font: 12px/1.5 "SF Mono", Menlo, Consolas, monospace; color: #2b2b2b; white-space: pre-wrap;
  box-shadow: 0 6px 10px rgba(0, 0, 0, .12);
}

.inspector h2 { font-size: 15px; }
.inspector .mono-list { font: 12.5px/1.6 "SF Mono", Menlo, Consolas, monospace; }
.inspector .mono-list .name { color: #1d5fa8; display: inline-block; min-width: 92px; }
.inspector pre { margin: 6px 0 0; font: 12px/1.5 "SF Mono", Menlo, Consolas, monospace; white-space: pre-wrap; word-break: break-
  word; }
.inspector .state { font: 600 13px "SF Mono", Menlo, Consolas, monospace; }

```

Листинг А.20 — atm-service/src/test/java/by/bsuir/bank/atm/AtmApiIT.java

```

package by.bsuir.bank.atm;

import by.bsuir.bank.atm.bank.BankGateway;
import by.bsuir.bank.atm.bank.BankReply;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

```

```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.ResultActions;

import java.util.Map;

import static org.hamcrest.Matchers.hasSize;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

/** Интеграционные тесты REST клиентского интерфейса банкомата; банк заменён заглушкой. */
@SpringBootTest
@AutoConfigureMockMvc
class AtmApiIT {

    @Autowired
    private MockMvc mvc;
    @Autowired
    private ObjectMapper json;
    @MockitoBean
    private BankGateway bank;

    @Test
    @DisplayName("Сеанс: открытие, ввод карты и PIN-кода, запрос остатка")
    void sessionFlow() throws Exception {
        when(bank.send(any())).thenReturn(
            new BankReply("OK", "OK", "Авторизация выполнена", Map.of("holder", "Иванов Иван Иванович")),
            new BankReply("OK", "OK", "Остаток", Map.of("account", "2400000010012", "balance", 750.5,
                "currency", "BYN", "bankDate", "2026-10-01")));
        String id = openSession();

        event(id, "enter", "{ \"value\": \"911238000000010\" }").andExpect(jsonPath("$.state").value("PIN"));
        event(id, "enter", "{ \"value\": \"1234\" }")
            .andExpect(jsonPath("$.state").value("MENU"))
            .andExpect(jsonPath("$.options", hasSize(5)))
            .andExpect(jsonPath("$.buffer[1].value").value("*****"));
        event(id, "select", "{ \"option\": \"BALANCE\" }").andExpect(jsonPath("$.state").value("RECEIPT_PROMPT"));
        event(id, "select", "{ \"option\": \"NO\" }")
            .andExpect(jsonPath("$.state").value("RESULT"))
            .andExpect(jsonPath("$.lines[0]").value("Доступно: 750.50 BYN"));

        mvc.perform(get("/api/sessions/" + id)).andExpect(jsonPath("$.state").value("RESULT"));
        mvc.perform(post("/api/sessions/" + id + "/cancel")).andExpect(jsonPath("$.state").value("MENU"));
    }

    @Test
    @DisplayName("Сеансы независимы друг от друга")
    void sessionsAreIsolated() throws Exception {
        String first = openSession();
        String second = openSession();

        event(first, "enter", "{ \"value\": \"911238000000010\" }").andExpect(jsonPath("$.state").value("PIN"));

        mvc.perform(get("/api/sessions/" + second)).andExpect(jsonPath("$.state").value("INSERT_CARD"));
    }

    @Test
    @DisplayName("Неизвестный сеанс — 404")
    void unknownSession() throws Exception {
        mvc.perform(get("/api/sessions/no-such-session"))
            .andExpect(status().isNotFound())
            .andExpect(jsonPath("$.code").value("NOT_FOUND"));
    }

    private String openSession() throws Exception {
        String response = mvc.perform(post("/api/sessions"))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.screen.state").value("INSERT_CARD"))
            .andReturn().getResponse().getContentAsString();
        return json.readTree(response).get("id").asText();
    }

    private ResultActions event(String id, String action, String body) throws Exception {
        return mvc.perform(post("/api/sessions/" + id + "/" + action).contentType(MediaType.APPLICATION_JSON).content(body))
    }

```

```

        .andExpect(status().isOk());
    }
}

```

Листинг А.21 –

atm-service/src/test/java/by/bsuir/bank/atm/AtmSessionTest.java

```

package by.bsuir.bank.atm;

import by.bsuir.bank.atm.bank.BankGateway;
import by.bsuir.bank.atm.bank.BankReply;
import by.bsuir.bank.atm.bank.Item;
import by.bsuir.bank.atm.bank.Operator;
import by.bsuir.bank.atm.bank.Transaction;
import by.bsuir.bank.atm.session.AtmSession;
import by.bsuir.bank.atm.session.Screen;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;
import org.mockito.ArgumentCaptor;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.mockito.junit.jupiter.MockitoSettings;
import org.mockito.quality.Strictness;
import org.springframework.web.client.ResourceAccessException;

import java.time.Clock;
import java.time.Instant;
import java.time.ZoneId;
import java.util.List;
import java.util.Map;

import static org.assertj.core.api.Assertions.assertThat;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.atLeastOnce;
import static org.mockito.Mockito.doReturn;
import static org.mockito.Mockito.doThrow;
import static org.mockito.Mockito.never;
import static org.mockito.Mockito.times;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;

/** Модульные тесты конечного автомата банкомата: банк заменён заглушкой, проверяются экраны и транзакции. */
@ExtendWith(MockitoExtension.class)
@MockitoSettings(strictness = Strictness.LENIENT)
class AtmSessionTest {

    static final String CARD = "9112380000000010";
    static final String PIN = "1234";

    @Mock
    private BankGateway bank;

    private AtmSession atm;

    @BeforeEach
    void bankAcceptsEverything() {
        Clock clock = Clock.fixed(Instant.parse("2026-10-01T09:30:00Z"), ZoneId.of("UTC"));
        atm = new AtmSession(bank, "ATM-TEST", clock);
        when(bank.send(any())).thenAnswer(call -> reply(call.getArgument(0)));
        when(bank.operators()).thenReturn(List.of(new Operator("A1", "A1"), new Operator("MTS", "MTC")));
    }

    /** Заглушка банка: PIN 1234 верный, на счёте 1000 BYN. */
    private static BankReply reply(Transaction tx) {
        Map<String, String> items = new java.util.LinkedHashMap<>();
        tx.items().forEach(item -> items.put(item.name(), item.value()));
        if (!PIN.equals(items.get(Item.PIN))) {
            return new BankReply("ERROR", "WRONG_PIN", "Неверный PIN-код", Map.of("attemptsLeft", 2));
        }
        return switch (items.get(Item.OPERATION)) {
            case "AUTHORIZE" -> ok("Авторизация выполнена", Map.of("holder", "Иванов Иван Иванович"));
            case "BALANCE" -> ok("Остаток", Map.of("account", "240000010012", "balance", 1000.0, "currency", "BYN",
                "bankDate", "2026-10-01"));
            case "DEPOSIT_BALANCE" -> ok("Вклады", Map.of("bankDate", "2026-10-01", "deposits", List.of(
                Map.of("number", "Д-000001", "product", "Свободное накопление", "amount", 1500, "currency", "BYN"))));
            case "WITHDRAW" -> Integer.parseInt(items.get(Item.AMOUNT)) > 1000

```

```

        ? new BankReply("ERROR", "INSUFFICIENT_FUNDS", "Недостаточно средств на счёте: доступно 1000.00 BYN", Map.of())
    )
    : ok("Заберите деньги", Map.of("amount", Integer.parseInt(items.get(Item.AMOUNT)),
        "balance", 1000 - Integer.parseInt(items.get(Item.AMOUNT)), "currency", "BYN", "bankDate", "2026-10-01"));
    case "PAYMENT" -> ok("Платёж принят", Map.of("operator", "МТС", "phone", items.get(Item.PHONE),
        "amount", Integer.parseInt(items.get(Item.AMOUNT)), "balance", 975, "currency", "BYN",
        "reference", "AB12CD34", "bankDate", "2026-10-01"));
    default -> new BankReply("ERROR", "UNKNOWN_OPERATION", "Операция не поддерживается", Map.of());
    };
}

private static BankReply ok(String message, Map<String, Object> data) {
    return new BankReply("OK", "OK", message, data);
}

@Test
@DisplayName("Начальный экран: банкомат ждёт карту")
void initialScreen() {
    Screen screen = atm.screen();

    assertThat(screen.state()).isEqualTo("INSERT_CARD");
    assertThat(screen.title()).isEqualTo("Вставьте, пожалуйста, карту");
    assertThat(screen.input().length()).isEqualTo(16);
    assertThat(screen.cardInside()).isFalse();
    assertThat(screen.buffer()).isEmpty();
}

@ParameterizedTest
@ValueSource(strings = {"", "123", "9112380000000001", "91123800000000011", "9112380000000001a"})
@DisplayName("Номер карты проверяется до обращения в банк: 16 цифр и контрольная цифра")
void cardNumberIsChecked(String number) {
    Screen screen = atm.enter(number);

    assertThat(screen.state()).isEqualTo("INSERT_CARD");
    assertThat(screen.notice()).isNotBlank();
    verify(bank, never()).send(any());
}

@Test
@DisplayName("Вход: номер карты и PIN-код попадают во внутренний список, банк авторизует карту")
void login() {
    Screen afterCard = atm.enter("9112 3800 0000 0010");
    assertThat(afterCard.state()).isEqualTo("PIN");
    assertThat(afterCard.cardInside()).isTrue();
    assertThat(afterCard.buffer()).containsExactly(new Item("CARD", CARD));

    Screen menu = atm.enter(PIN);
    assertThat(menu.state()).isEqualTo("MENU");
    assertThat(menu.options()).extracting(Screen.Option::key)
        .containsExactly("WITHDRAW", "BALANCE", "DEPOSIT_BALANCE", "PAYMENT", "EJECT");
    // PIN-код на экран и в журнал обмена выводится звёздочками
    assertThat(menu.buffer()).containsExactly(new Item("CARD", CARD), new Item("PIN", "*****"));
    assertThat(menu.exchange().request()).extracting(Item::value).containsExactly(CARD, "*****", "AUTHORIZE");
    assertThat(sent().items()).containsExactly(new Item("CARD", CARD), new Item("PIN", PIN), new Item("OPERATION", "AUTHORIZE")
    ));
}

@Test
@DisplayName("Неверный PIN-код запрашивается повторно, после трёх ошибок работа завершается")
void threeWrongPins() {
    atm.enter(CARD);

    Screen first = atm.enter("0000");
    assertThat(first.state()).isEqualTo("PIN");
    assertThat(first.notice()).isEqualTo("Неверный PIN-код. Осталось попыток: 2");
    assertThat(first.buffer()).containsExactly(new Item("CARD", CARD));

    assertThat(atm.enter("1111").notice()).isEqualTo("Неверный PIN-код. Осталось попыток: 1");

    Screen ejected = atm.enter("2222");
    assertThat(ejected.state()).isEqualTo("INSERT_CARD");
    assertThat(ejected.notice()).startsWith("PIN-код введён неверно три раза");
    assertThat(ejected.cardInside()).isFalse();
    assertThat(ejected.buffer()).isEmpty();
    verify(bank, times(3)).send(any());
}

@Test
@DisplayName("PIN-код из трёх цифр в банк не отправляется")
void shortPinIsRejectedLocally() {
    atm.enter(CARD);

```

```

        assertThat(atm.enter("123").notice()).isEqualTo("PIN-код состоит из 4 цифр");
        verify(bank, never()).send(any());
    }

    @Test
    @DisplayName("Заблокированная карта возвращается клиенту")
    void blockedCardIsEjected() {
        doReturn(new BankReply("ERROR", "CARD_BLOCKED", "Карта заблокирована. Обратитесь в банк", Map.of()))
            .when(bank).send(any());
        atm.enter(CARD);

        Screen screen = atm.enter(PIN);

        assertThat(screen.state()).isEqualTo("INSERT_CARD");
        assertThat(screen.notice()).contains("Карта заблокирована");
    }

    @Test
    @DisplayName("Снятие наличных: список уходит банку целиком и очищается, по желанию печатается чек")
    void withdrawal() {
        login(atm);

        assertThat(atm.select("WITHDRAW").state()).isEqualTo("AMOUNT");
        Screen done = atm.enter("300");

        assertThat(sent().items()).containsExactly(new Item("CARD", CARD), new Item("PIN", PIN),
            new Item("OPERATION", "WITHDRAW"), new Item("AMOUNT", "300"));
        assertThat(done.state()).isEqualTo("RECEIPT_PROMPT");
        assertThat(done.title()).isEqualTo("Заберите деньги");
        assertThat(done.cash()).isEqualTo("300.00 BYN");
        assertThat(done.buffer()).isEmpty();

        Screen menu = atm.select("YES");
        assertThat(menu.state()).isEqualTo("MENU");
        assertThat(menu.receipt()).contains("ВЫДАЧА НАЛИЧНЫХ", "Сумма: 300.00 BYN", "Остаток: 700.00 BYN",
            "Карта **** * 0010", "Банкомат АТМ-ТЕСТ", "Банковский день: 01.10.2026");
        assertThat(menu.cash()).isNull();
    }

    @ParameterizedTest
    @ValueSource(strings = {"", "0", "000", "12a", "-5", "1.5", "12345678"})
    @DisplayName("Сумма – целое положительное число; иначе запрос в банк не отправляется")
    void amountIsChecked(String amount) {
        login(atm);
        atm.select("WITHDRAW");

        Screen screen = atm.enter(amount);

        assertThat(screen.state()).isEqualTo("AMOUNT");
        assertThat(screen.notice()).isNotBlank();
        verify(bank, times(1)).send(any()); // только авторизация
    }

    @Test
    @DisplayName("Недостаточно средств: показывается сообщение банка, затем главное меню")
    void insufficientFunds() {
        login(atm);
        atm.select("WITHDRAW");

        Screen message = atm.enter("5000");
        assertThat(message.state()).isEqualTo("MESSAGE");
        assertThat(message.title()).isEqualTo("Операция не выполнена");
        assertThat(message.notice()).startsWith("Недостаточно средств");
        assertThat(message.cash()).isNull();

        assertThat(atm.select("CONTINUE").state()).isEqualTo("MENU");
    }

    @Test
    @DisplayName("Перед следующей операцией PIN-код вводится заново, номер карты банкомат помнит сам")
    void secondOperationRequiresPinAgain() {
        login(atm);
        atm.select("WITHDRAW");
        atm.enter("100");
        atm.select("NO");

        Screen pin = atm.select("BALANCE");
        assertThat(pin.state()).isEqualTo("PIN");
        assertThat(pin.title()).isEqualTo("Введите PIN-код ещё раз");
        assertThat(pin.buffer()).containsExactly(new Item("CARD", CARD));

        Screen prompt = atm.enter(PIN);
    }

```

```

        assertThat(prompt.state()).isEqualTo("RECEIPT_PROMPT");
        assertThat(sent().items()).containsExactly(new Item("CARD", CARD), new Item("PIN", PIN), new Item("OPERATION", "BALANCE"));
    }

    @Test
    @DisplayName("Остаток счёта: вопрос о чеке, затем состояние счёта и выбор — продолжить или забрать карту")
    void balanceInquiry() {
        login(atm);

        Screen prompt = atm.select("BALANCE");
        assertThat(prompt.state()).isEqualTo("RECEIPT_PROMPT");
        assertThat(prompt.lines()).containsExactly("Распечатать чек?");

        Screen result = atm.select("YES");
        assertThat(result.state()).isEqualTo("RESULT");
        assertThat(result.lines()).containsExactly("Доступно: 1000.00 BYN");
        assertThat(result.receipt()).contains("ОСТАТОК КРЕДИТНОГО СЧЁТА", "Счёт: 2400000010012", "Доступно: 1000.00 BYN");
        assertThat(result.options().extracting(Screen.Option::key).containsExactly("CONTINUE", "EJECT"));

        Screen ejected = atm.select("EJECT");
        assertThat(ejected.state()).isEqualTo("INSERT_CARD");
        assertThat(ejected.cardInside()).isFalse();
    }

    @Test
    @DisplayName("Остаток депозитного счёта: без чека сразу показываются вклады")
    void depositBalance() {
        login(atm);
        atm.select("DEPOSIT_BALANCE");

        Screen result = atm.select("NO");

        assertThat(result.state()).isEqualTo("RESULT");
        assertThat(result.title()).isEqualTo("Депозитные счета");
        assertThat(result.lines()).containsExactly("Д-000001: 1500.00 BYN");
        assertThat(result.receipt()).isNull();
        assertThat(atm.select("CONTINUE").state()).isEqualTo("MENU");
    }

    @Test
    @DisplayName("Платёж: оператор, номер, сумма, подтверждение; чек печатается автоматически")
    void payment() {
        login(atm);

        Screen operators = atm.select("PAYMENT");
        assertThat(operators.state()).isEqualTo("OPERATOR");
        assertThat(operators.options().extracting(Screen.Option::label).containsExactly("A1", "MTC");

        assertThat(atm.select("MTS").state()).isEqualTo("PHONE");
        assertThat(atm.enter("029765432").notice()).startsWith("Номер телефона — 10 цифр");
        assertThat(atm.enter("0297654321").state()).isEqualTo("PAY_AMOUNT");

        Screen confirm = atm.enter("25");
        assertThat(confirm.state()).isEqualTo("CONFIRM");
        assertThat(confirm.lines()).containsExactly("Оператор: MTC", "Телефон: 0297654321", "Сумма: 25 BYN");
        verify(bank, times(1)).send(any()); // до подтверждения в банк ушла только авторизация

        Screen done = atm.select("CONFIRM");
        assertThat(sent().items()).containsExactly(new Item("CARD", CARD), new Item("PIN", PIN),
            new Item("OPERATION", "PAYMENT"), new Item("OPERATOR", "MTS"),
            new Item("PHONE", "0297654321"), new Item("AMOUNT", "25"));
        assertThat(done.state()).isEqualTo("MESSAGE");
        assertThat(done.title()).isEqualTo("Платёж принят");
        assertThat(done.receipt()).contains("ОПЛАТА УСЛУГ СВЯЗИ", "Оператор: MTC", "Телефон: 0297654321",
            "Сумма: 25.00 BYN", "Операция № AB12CD34");
        assertThat(done.buffer()).isEmpty();
    }

    @Test
    @DisplayName("Платёж: «Ввести заново» стирает оператора, номер и сумму, но не данные карты")
    void paymentRetry() {
        login(atm);
        atm.select("PAYMENT");
        atm.select("A1");
        atm.enter("0291112233");
        atm.enter("10");

        Screen again = atm.select("RETRY");

        assertThat(again.state()).isEqualTo("OPERATOR");
        assertThat(again.buffer().extracting(Item::name).containsExactly("CARD", "PIN", "OPERATION"));
    }

```

```

    }

    @Test
    @DisplayName("«Отмена» внутри операции возвращает в меню, в меню — возвращает карту")
    void cancel() {
        login(atm);
        atm.select("WITHDRAW");

        Screen menu = atm.cancel();
        assertThat(menu.state()).isEqualTo("MENU");
        assertThat(menu.notice()).isEqualTo("Операция отменена");
        assertThat(menu.buffer().extracting(Item::name).containsExactly("CARD", "PIN"));

        Screen ejected = atm.cancel();
        assertThat(ejected.state()).isEqualTo("INSERT_CARD");
        assertThat(ejected.buffer()).isEmpty();
    }

    @Test
    @DisplayName("Нет связи с банком: авторизация не проходит, PIN-код из списка удаляется")
    void bankIsUnavailable() {
        doThrow(new ResourceAccessException("Connection refused")).when(bank).send(any());
        atm.enter(CARD);

        Screen screen = atm.enter(PIN);

        assertThat(screen.state()).isEqualTo("PIN");
        assertThat(screen.notice()).startsWith("Нет связи с банком");
        assertThat(screen.buffer()).containsExactly(new Item("CARD", CARD));
        assertThat(screen.exchange().reply()).isNull();
    }

    @Test
    @DisplayName("Забрать карту: сеанс завершается, банкомат ждёт следующую карту")
    void eject() {
        login(atm);

        Screen screen = atm.select("EJECT");

        assertThat(screen.state()).isEqualTo("INSERT_CARD");
        assertThat(screen.notice()).startsWith("Заберите карту");
        assertThat(screen.cardInside()).isFalse();
        assertThat(screen.buffer()).isEmpty();
    }

    static void login(AtmSession atm) {
        atm.enter(CARD);
        atm.enter(PIN);
    }

    private Transaction sent() {
        ArgumentCaptor<Transaction> captor = ArgumentCaptor.forClass(Transaction.class);
        verify(bank, atLeastOnce()).send(captor.capture());
        return captor.getValue();
    }
}

```

Листинг A.22 — atm-service/src/test/java/by/bsuir/bank/atm/AtmUIT.java

```

package by.bsuir.bank.atm;

import by.bsuir.bank.atm.bank.BankGateway;
import by.bsuir.bank.atm.bank.BankReply;
import by.bsuir.bank.atm.bank.Operator;
import org.junit.jupiter.api.AfterAll;
import org.junit.jupiter.api.Assumptions;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.openqa.selenium.By;
import org.openqa.selenium.Keys;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.test.web.server.LocalServerPort;
import org.springframework.test.context.bean.override.mockito.MockitoBean;

```

```

import java.time.Duration;
import java.util.List;
import java.util.Map;

import static org.assertj.core.api.Assertions.assertThat;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.when;

/** Функциональные тесты интерфейса банкомата через WebDriver; банк заменён заглушкой. */
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
class AtmUiIT {

    private static final String PIN = "1234";

    private static WebDriver driver;

    @LocalServerPort
    private int port;
    @MockitoBean
    private BankGateway bank;

    private WebDriverWait wait;

    @BeforeAll
    static void startBrowser() {
        try {
            ChromeOptions options = new ChromeOptions();
            options.addArguments("--headless=new", "--window-size=1400,1000", "--lang=ru");
            driver = new ChromeDriver(options);
        } catch (Exception e) {
            Assumptions.abort("Chrome недоступен, функциональные тесты пропущены: " + e.getMessage());
        }
    }

    @AfterAll
    static void stopBrowser() {
        if (driver != null) {
            driver.quit();
        }
    }

    @BeforeEach
    void openAtm() {
        when(bank.send(any())).thenAnswer(call -> AtmSessionTestBank.reply(call.getArgument(0)));
        when(bank.operators()).thenReturn(List.of(new Operator("A1", "A1"), new Operator("MTS", "MTC")));
        wait = new WebDriverWait(driver, Duration.ofSeconds(10));
        driver.get("http://localhost:" + port + "/");
        waitForState("INSERT_CARD");
    }

    @Test
    @DisplayName("Клиент вставляет карту, вводит PIN-код, снимает деньги с чеком и забирает карту")
    void withdrawWithReceipt() {
        press("9112380000000010");
        assertThat(driver.findElement(By.id("display-input")).getText()).isEqualTo("9112 3800 0000 0010");
        click("key-enter");
        waitForState("PIN");

        press(PIN);
        assertThat(driver.findElement(By.id("display-input")).getText()).isEqualTo("●●●●");
        click("key-enter");
        waitForState("MENU");
        assertThat(driver.findElement(By.id("buffer")).getText()).contains("CARD", "PIN", "*****").doesNotContain(PIN);

        click("opt-WITHDRAW");
        waitForState("AMOUNT");
        press("250");
        click("key-enter");
        waitForState("RECEIPT_PROMPT");
        assertThat(driver.findElement(By.id("display-title")).getText()).isEqualTo("Заберите деньги");
        assertThat(driver.findElement(By.id("cash")).getText()).isEqualTo("250.00 BYN");

        click("opt-YES");
        waitForState("MENU");
        assertThat(driver.findElement(By.id("receipt")).getText())
            .contains("ВЫДАЧА НАЛИЧНЫХ", "Сумма: 250.00 BYN", "Остаток: 750.00 BYN");

        click("opt-EJECT");
        waitForState("INSERT_CARD");
        assertThat(driver.findElement(By.id("notice")).getText()).startsWith("Заберите карту");
    }
}

```

```

@Test
@DisplayName("Три неверных PIN-кода с клавиатуры – работа с банкоматом завершается")
void threeWrongPins() {
    driver.findElement(By.tagName("body")).sendKeys("9112380000000010", Keys.ENTER);
    waitForState("PIN");

    for (int attempt = 2; attempt >= 1; attempt--) {
        driver.findElement(By.tagName("body")).sendKeys("0000", Keys.ENTER);
        wait.until(ExpectedConditions.textToBe(By.id("notice"), "Неверный PIN-код. Осталось попыток: " + attempt));
    }
    driver.findElement(By.tagName("body")).sendKeys("0000", Keys.ENTER);

    waitForState("INSERT_CARD");
    assertThat(driver.findElement(By.id("notice")).getText()).startsWith("PIN-код введён неверно три раза");
}

@Test
@DisplayName("Оплата мобильной связи с подтверждением данных и автоматической печатью чека")
void payment() {
    press("9112380000000010");
    click("key-enter");
    waitForState("PIN");
    press(PIN);
    click("key-enter");
    waitForState("MENU");

    click("opt-PAYMENT");
    waitForState("OPERATOR");
    click("opt-MTS");
    waitForState("PHONE");
    press("0297654321");
    click("key-enter");
    waitForState("PAY_AMOUNT");
    press("25");
    click("key-enter");
    waitForState("CONFIRM");
    assertThat(driver.findElement(By.id("display-lines")).getText())
        .contains("Оператор: МТС", "Телефон: 0297654321", "Сумма: 25 BYN");

    click("opt-CONFIRM");
    waitForState("MESSAGE");
    assertThat(driver.findElement(By.id("display-title")).getText()).isEqualTo("Платёж принят");
    assertThat(driver.findElement(By.id("receipt")).getText()).contains("ОПЛАТА УСЛУГ СВЯЗИ", "Операция № AB12CD34");
}

private void press(String digits) {
    for (char digit : digits.toCharArray()) {
        click("key-" + digit);
    }
}

private void click(String id) {
    wait.until(ExpectedConditions.elementToBeClickable(By.id(id))).click();
}

private void waitForState(String state) {
    wait.until(ExpectedConditions.textToBe(By.id("state"), state));
    wait.until(ExpectedConditions.attributeToBe(By.id("display"), "data-state", state));
}

/** Та же заглушка банка, что и в модульных тестах автомата. */
static final class AtmSessionTestBank {

    static BankReply reply(by.bsuir.bank.atm.bank.Transaction tx) {
        Map<String, String> items = new java.util.LinkedHashMap<>();
        tx.items().forEach(item -> items.put(item.name(), item.value()));
        if (!PIN.equals(items.get("PIN"))) {
            return new BankReply("ERROR", "WRONG_PIN", "Неверный PIN-код", Map.of());
        }
        return switch (items.get("OPERATION")) {
            case "WITHDRAW" -> new BankReply("OK", "OK", "Заберите деньги", Map.of(
                "amount", Integer.parseInt(items.get("AMOUNT")),
                "balance", 1000 - Integer.parseInt(items.get("AMOUNT")), "currency", "BYN", "bankDate", "2026-10-01"));
            case "PAYMENT" -> new BankReply("OK", "OK", "Платёж принят", Map.of("operator", "МТС",
                "phone", items.get("PHONE"), "amount", Integer.parseInt(items.get("AMOUNT")), "balance", 975,
                "currency", "BYN", "reference", "AB12CD34", "bankDate", "2026-10-01"));
            default -> new BankReply("OK", "OK", "Авторизация выполнена", Map.of("holder", "Иванов Иван Иванович"));
        };
    }
}
}

```

Банковская сторона протокола (credit-service)

Листинг А.23 –

credit-service/src/main/java/by/bsuir/bank/credit/domain/Card.java

```
package by.bsuir.bank.credit.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.OneToOne;
import lombok.Getter;
import lombok.Setter;

/** Банковская карта, привязанная к кредитному счёту договора. */
@Entity
@Getter
@Setter
public class Card {

    public static final int MAX_PIN_ATTEMPTS = 3;

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String number;

    @OneToOne(fetch = FetchType.EAGER, optional = false)
    private CreditContract contract;

    private String holder;
    private String pinHash;
    private int failedAttempts;
    private boolean blocked;

    /** Неверный PIN-код: после третьей ошибки подряд карта блокируется. Возвращает число оставшихся попыток. */
    public int registerFailedAttempt() {
        failedAttempts++;
        if (failedAttempts >= MAX_PIN_ATTEMPTS) {
            blocked = true;
        }
        return Math.max(0, MAX_PIN_ATTEMPTS - failedAttempts);
    }

    public void resetAttempts() {
        failedAttempts = 0;
        blocked = false;
    }
}
```

Листинг А.24 –

credit-service/src/main/java/by/bsuir/bank/credit/domain/CardNumbers.java

```
package by.bsuir.bank.credit.domain;

/** Номер карты: 16 цифр, префикс банка, порядковый номер и контрольная цифра по алгоритму Луна. */
public final class CardNumbers {

    private static final String BIN = "911238";

    public static String generate(long sequence) {
        String body = BIN + String.format("%09d", sequence);
        return body + luhnDigit(body);
    }

    public static boolean isValid(String number) {
        return number != null && number.matches("\\d{16}")
            && luhnDigit(number.substring(0, 15)) == number.charAt(15) - '0';
    }

    /** Цифры на чётных с конца позициях удваиваются; контрольная цифра дополняет сумму до кратной 10. */
    static int luhnDigit(String body) {
        int sum = 0;
        boolean doubled = true;
        for (int i = body.length() - 1; i >= 0; i--) {
            int digit = body.charAt(i) - '0';
            if (doubled) {
                digit *= 2;
                if (digit > 9) {
                    digit -= 9;
                }
            }
            sum += digit;
            doubled = !doubled;
        }
        return (sum + 1) % 10;
    }
}
```

```

        for (int i = body.length() - 1; i >= 0; i--) {
            int digit = body.charAt(i) - '0';
            if (doubled) {
                digit *= 2;
                if (digit > 9) {
                    digit -= 9;
                }
            }
            sum += digit;
            doubled = !doubled;
        }
        return (10 - sum % 10) % 10;
    }

    private CardNumbers() {
    }
}

```

Листинг A.25 –

credit-service/src/main/java/by/bsuir/bank/credit/domain/MobileOperator.java

```

package by.bsuir.bank.credit.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import lombok.Getter;
import lombok.Setter;

/** Оператор мобильной связи – получатель платежей из банкомата. */
@Entity
@Getter
@Setter
public class MobileOperator {

    @Id
    private String code;

    private String name;
}

```

Листинг A.26 –

credit-service/src/main/java/by/bsuir/bank/credit/domain/MobilePayment.java

```

package by.bsuir.bank.credit.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;
import java.time.LocalDate;

/** Платёж за услуги мобильной связи, проведённый через банкомат. */
@Entity
@Getter
@Setter
public class MobilePayment {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String cardNumber;
    private String operator;
    private String phone;
    private BigDecimal amount;
    private LocalDate bankDate;
    private String reference;
}

```

Листинг A.27 –

credit-service/src/main/java/by/bsuir/bank/credit/dto/PinEnvelope.java

```
package by.bsuir.bank.credit.dto;

/** «ПИН-конверт»: номер карты и PIN-код в открытом виде. В базе остаётся только хеш PIN-кода. */
public record PinEnvelope(String cardNumber, String pin) {
}
```

Листинг А.28 – credit-service/src/main/java/by/bsuir/bank/credit/service/CardService.java

```
package by.bsuir.bank.credit.service;

import by.bsuir.bank.common.api.BankException;
import by.bsuir.bank.credit.domain.Card;
import by.bsuir.bank.credit.domain.CardNumbers;
import by.bsuir.bank.credit.domain.CreditContract;
import by.bsuir.bank.credit.dto.PinEnvelope;
import by.bsuir.bank.credit.repository.CardRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.security.SecureRandom;

/** Выпуск карт к кредитным счетам и проверка PIN-кодов. */
@Service
@RequiredArgsConstructor
public class CardService {

    private final CardRepository cards;
    private final PasswordEncoder encoder = new BCryptPasswordEncoder();
    private final SecureRandom random = new SecureRandom();

    /** Выпускает карту к договору. PIN-код генерируется случайно и возвращается единственный раз. */
    @Transactional
    public PinEnvelope issue(CreditContract contract) {
        Card card = new Card();
        card.setNumber(CardNumbers.generate(contract.getId()));
        card.setContract(contract);
        card.setHolder(contract.getClientName());
        return assignPin(card);
    }

    /** Перевыпуск PIN-кода: заодно снимает блокировку карты после трёх неверных попыток. */
    @Transactional
    public PinEnvelope reissuePin(Long contractId) {
        Card card = cards.findByContractId(contractId)
            .orElseThrow(() -> BankException.notFound("К договору не выпущена карта"));
        card.resetAttempts();
        return assignPin(card);
    }

    public boolean matches(Card card, String pin) {
        return pin != null && encoder.matches(pin, card.getPinHash());
    }

    private PinEnvelope assignPin(Card card) {
        String pin = String.format("%04d", random.nextInt(10_000));
        card.setPinHash(encoder.encode(pin));
        cards.save(card);
        return new PinEnvelope(card.getNumber(), pin);
    }
}
```

Листинг А.29 – credit-service/src/main/java/by/bsuir/bank/credit/atm/AtmConfig.java

```
package by.bsuir.bank.credit.atm;

import by.bsuir.bank.common.HttpApis;
import by.bsuir.bank.common.ServiceUrls;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.client.RestClient;

@Configuration
```

```

public class AtmConfig {

    @Bean
    public DepositApi depositApi(RestClient.Builder builder, ServiceUrls urls) {
        return HttpApis.proxy(builder, urls.deposits(), DepositApi.class);
    }
}

```

Листинг А.30 – credit-service/src/main/java/by/bsuir/bank/credit/atm/AtmProcessor.java

```

package by.bsuir.bank.credit.atm;

import by.bsuir.bank.common.api.ApiError;
import by.bsuir.bank.common.api.BankException;
import by.bsuir.bank.common.ledger.AccountInfo;
import by.bsuir.bank.common.ledger.Batch;
import by.bsuir.bank.common.ledger.LedgerApi;
import by.bsuir.bank.common.ledger.OpenAccount;
import by.bsuir.bank.credit.domain.Card;
import by.bsuir.bank.credit.domain.CreditContract;
import by.bsuir.bank.credit.domain.MobileOperator;
import by.bsuir.bank.credit.domain.MobilePayment;
import by.bsuir.bank.credit.repository.CardRepository;
import by.bsuir.bank.credit.repository.MobileOperatorRepository;
import by.bsuir.bank.credit.repository.MobilePaymentRepository;
import by.bsuir.bank.credit.service.CardService;
import by.bsuir.bank.credit.service.CreditPostings;
import by.bsuir.bank.credit.service.CreditService;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.client.RestClientException;
import org.springframework.web.client.RestClientResponseException;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.UUID;

/**
 * Банковская сторона протокола «банк – банкомат». Каждая транзакция банкомата содержит номер карты,
 * PIN-код и описание операции; банк заново проверяет карту и PIN-код при каждом запросе.
 */
@Service
@RequiredArgsConstructor
public class AtmProcessor {

    public static final String OPERATOR_CHART_CODE = "3012";

    private static final String PHONE_FORMAT = "^0(25|29|33|44)\\d{7}$";

    private final CardRepository cards;
    private final CardService cardService;
    private final CreditService credits;
    private final MobileOperatorRepository operators;
    private final MobilePaymentRepository payments;
    private final LedgerApi ledger;
    private final DepositApi deposits;

    @Transactional
    public AtmResponse handle(AtmRequest request) {
        Map<String, String> tx = new LinkedHashMap<>();
        request.items().forEach(item -> tx.put(item.name(), item.value()));
        String operation = tx.get("OPERATION");
        if (tx.get("CARD") == null || tx.get("PIN") == null || operation == null) {
            return AtmResponse.error("BAD_REQUEST", "Неполное описание транзакции");
        }

        Card card = cards.findByNumber(tx.get("CARD")).orElse(null);
        if (card == null) {
            return AtmResponse.error("CARD_NOT_FOUND", "Карта не обслуживается банком");
        }
        if (card.isBlocked()) {
            return AtmResponse.error("CARD_BLOCKED", "Карта заблокирована. Обратитесь в банк");
        }
        if (!cardService.matches(card, tx.get("PIN"))) {
            int left = card.registerFailedAttempt();

```

```

        return left == 0
            ? AtmResponse.error("CARD_BLOCKED", "PIN-код трижды введён неверно. Карта заблокирована")
            : AtmResponse.error("WRONG_PIN", "Неверный PIN-код. Осталось попыток: " + left, Map.of("attemptsLeft", left));
    }
    card.setFailedAttempts(0);

    try {
        return switch (operation) {
            case "AUTHORIZE" -> AtmResponse.ok("Авторизация выполнена", Map.of("holder", card.getHolder()));
            case "BALANCE" -> balance(card);
            case "DEPOSIT_BALANCE" -> depositBalance(card);
            case "WITHDRAW" -> withdraw(card, tx.get("AMOUNT"));
            case "PAYMENT" -> payment(card, tx.get("OPERATOR"), tx.get("PHONE"), tx.get("AMOUNT"));
            default -> AtmResponse.error("UNKNOWN_OPERATION", "Операция не поддерживается");
        };
    } catch (BankException e) {
        return AtmResponse.error(e.getCode(), e.getMessage());
    } catch (RestClientResponseException e) {
        return rejected(e);
    } catch (RestClientException e) {
        return AtmResponse.error("SERVICE_UNAVAILABLE", "Операция временно недоступна");
    }
}

/** Отказ смежного сервиса (например, главной книги) передаётся банкомату с исходным кодом и текстом. */
private static AtmResponse rejected(RestClientResponseException e) {
    try {
        ApiError error = e.getResponseBodyAs(ApiError.class);
        if (error != null && error.code() != null && error.message() != null) {
            return AtmResponse.error(error.code(), error.message());
        }
    } catch (RuntimeException ignored) {
        // тело ответа не в формате ApiError
    }
    return AtmResponse.error("BANK_ERROR", "Операция отклонена банком");
}

public List<MobileOperator> operators() {
    return operators.findAll();
}

private AtmResponse balance(Card card) {
    CreditContract contract = card.getContract();
    AccountInfo account = ledger.account(contract.getMainAccount());
    return AtmResponse.ok("Остаток кредитного счёта", receipt(card, Map.of(
        "account", account.number(), "balance", account.balance(), "currency", account.currency())));
}

private AtmResponse depositBalance(Card card) {
    List<Map<String, Object>> active = deposits.byClient(card.getContract().getClientId()).stream()
        .filter(deposit -> "ACTIVE".equals(deposit.status()))
        .map(deposit -> Map.of("number", deposit.number(), "product", deposit.productName(),
            "amount", deposit.amount(), "currency", deposit.currency()))
        .toList();
    return AtmResponse.ok(active.isEmpty() ? "Действующих вкладов нет" : "Остатки депозитных счетов",
        receipt(card, Map.of("deposits", active)));
}

private AtmResponse withdraw(Card card, String text) {
    if (text == null || !text.matches("\\d{1,9}") || Long.parseLong(text) == 0) {
        return AtmResponse.error("INVALID_AMOUNT", "Сумма должна быть целым положительным числом");
    }
    BigDecimal amount = new BigDecimal(text);
    AccountInfo account = credits.cashOut(card.getContract().getId(), amount);
    return AtmResponse.ok("Заберите деньги", receipt(card, Map.of(
        "amount", amount, "balance", account.balance(), "currency", account.currency())));
}

private AtmResponse payment(Card card, String operatorCode, String phone, String text) {
    MobileOperator operator = operatorCode == null ? null : operators.findById(operatorCode).orElse(null);
    if (operator == null) {
        return AtmResponse.error("UNKNOWN_OPERATOR", "Оператор связи не найден");
    }
    if (phone == null || !phone.matches(PHONE_FORMAT)) {
        return AtmResponse.error("INVALID_PHONE", "Номер телефона: 10 цифр, например 0291234567");
    }
    if (text == null || !text.matches("\\d{1,9}(\\.\\d{1,2})?") || new BigDecimal(text).signum() == 0) {
        return AtmResponse.error("INVALID_AMOUNT", "Сумма платежа должна быть положительным числом");
    }
    BigDecimal amount = new BigDecimal(text);
    CreditContract contract = card.getContract();
    String currency = contract.getCurrency();

```

```

AccountInfo main = ledger.account(contract.getMainAccount());
if (main.balance().compareTo(amount) < 0) {
    return AtmResponse.error("INSUFFICIENT_FUNDS",
        "Недостаточно средств на счёте: доступно " + main.balance() + " " + currency);
}

// расчётный счёт оператора открывается при первом платеже; повторный запрос вернёт тот же счёт
String operatorAccount = ledger.open(new OpenAccount(OPERATOR_CHART_CODE, currency, null,
    "Расчётный счёт оператора " + operator.getName(), "OPERATOR-" + operator.getCode() + "-" + currency)).number();
String reference = UUID.randomUUID().toString().substring(0, 8).toUpperCase();
LocalDate date = ledger.bankDay().date();
ledger.post(Batch.of("ATM-PAYMENT-" + reference, contract.getNumber(), date,
    CreditPostings.mobilePayment(main.number(), operatorAccount,
        "Оплата услуг связи " + operator.getName() + ", номер " + phone, amount)));

MobilePayment payment = new MobilePayment();
payment.setCardNumber(card.getNumber());
payment.setOperator(operator.getCode());
payment.setPhone(phone);
payment.setAmount(amount);
payment.setBankDate(date);
payment.setReference(reference);
payments.save(payment);

return AtmResponse.ok("Платёж принят", receipt(card, Map.of(
    "operator", operator.getName(), "phone", phone, "amount", amount, "currency", currency,
    "balance", main.balance().subtract(amount), "reference", reference)));
}

/** Общие реквизиты для чека: банковский день и держатель карты. */
private Map<String, Object> receipt(Card card, Map<String, Object> details) {
    Map<String, Object> data = new LinkedHashMap<>(details);
    data.put("bankDate", ledger.bankDay().date().toString());
    data.put("holder", card.getHolder());
    return data;
}
}

```

Листинг А.31 – credit-service/src/main/java/by/bsuir/bank/credit/atm/AtmRequest.java

```

package by.bsuir.bank.credit.atm;

import jakarta.validation.constraints.NotEmpty;

import java.util.List;

/**
 * Запрос банкомата – содержимое его «внутреннего списка»: упорядоченные пары «имя – значение».
 * Имена элементов: CARD, PIN, OPERATION, AMOUNT, OPERATOR, PHONE.
 */
public record AtmRequest(@NotEmpty List<Item> items) {

    public record Item(String name, String value) {
    }
}

```

Листинг А.32 – credit-service/src/main/java/by/bsuir/bank/credit/atm/AtmResponse.java

```

package by.bsuir.bank.credit.atm;

import java.util.Map;

/** Ответ банка банкомату: статус OK/ERROR, код и текст для экрана, данные для чека. */
public record AtmResponse(String status, String code, String message, Map<String, Object> data) {

    public static AtmResponse ok(String message, Map<String, Object> data) {
        return new AtmResponse("OK", "OK", message, data);
    }

    public static AtmResponse error(String code, String message) {
        return new AtmResponse("ERROR", code, message, Map.of());
    }

    public static AtmResponse error(String code, String message, Map<String, Object> data) {
        return new AtmResponse("ERROR", code, message, data);
    }
}

```

```
}
```

Листинг А.33 – credit-service/src/main/java/by/bsuir/bank/credit/atm/DepositApi.java

```
package by.bsuir.bank.credit.atm;

import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.service.annotation.GetExchange;
import org.springframework.web.service.annotation.HttpExchange;

import java.math.BigDecimal;
import java.util.List;

/** REST-клиент сервиса депозитов: вклады клиента для операции банкомата «Остаток депозитного счёта». */
@HttpExchange("/api/deposits")
public interface DepositApi {

    record DepositInfo(String number, String productName, String currency, BigDecimal amount, String status) {
    }

    @GetExchange
    List<DepositInfo> byClient(@RequestParam Long clientId);
}
```

Листинг А.34 – credit-service/src/main/java/by/bsuir/bank/credit/web/AtmController.java

```
package by.bsuir.bank.credit.web;

import by.bsuir.bank.credit.atm.AtmProcessor;
import by.bsuir.bank.credit.atm.AtmRequest;
import by.bsuir.bank.credit.atm.AtmResponse;
import by.bsuir.bank.credit.domain.MobileOperator;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

/** Точка входа протокола «банк – банкомат». */
@RestController
@RequestMapping("/api/atm")
@RequiredArgsConstructor
public class AtmController {

    private final AtmProcessor processor;

    @PostMapping("/transactions")
    public AtmResponse transaction(@Valid @RequestBody AtmRequest request) {
        return processor.handle(request);
    }

    @GetMapping("/operators")
    public List<MobileOperator> operators() {
        return processor.operators();
    }
}
```

Листинг А.35 – credit-service/src/test/java/by/bsuir/bank/credit/AtmProtocolIT.java

```
package by.bsuir.bank.credit;

import by.bsuir.bank.common.client.ClientApi;
import by.bsuir.bank.common.client.ClientInfo;
import by.bsuir.bank.common.ledger.AccountInfo;
import by.bsuir.bank.common.ledger.InMemoryLedger;
import by.bsuir.bank.common.ledger.LedgerApi;
import by.bsuir.bank.common.ledger.OperationInfo;
import by.bsuir.bank.credit.atm.DepositApi;
```

```

import by.bsuir.bank.credit.atm.DepositApi.DepositInfo;
import by.bsuir.bank.credit.dto.CreditIssued;
import by.bsuir.bank.credit.dto.CreditRequest;
import by.bsuir.bank.credit.repository.CardRepository;
import by.bsuir.bank.credit.repository.CreditContractRepository;
import by.bsuir.bank.credit.repository.MobilePaymentRepository;
import by.bsuir.bank.credit.service.CreditService;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import org.junit.jupiter.params.provider.ValueSource;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.context.annotation.Import;
import org.springframework.http.MediaType;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.ResultActions;
import org.springframework.web.client.ResourceAccessException;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;

import static by.bsuir.bank.common.ledger.InMemoryLedger.START;
import static org.assertj.core.api.Assertions.assertThat;
import static org.hamcrest.Matchers.hasSize;
import static org.mockito.ArgumentMatchers.anyLong;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

/** Интеграционные тесты банковской стороны протокола «банк – банкомат». */
@SpringBootTest(properties = "spring.datasource.url=jdbc:h2:mem:${random.uuid};DB_CLOSE_DELAY=-1")
@AutoConfigureMockMvc
@Import(TestLedgerConfig.class)
class AtmProtocolIT {

    @Autowired
    private MockMvc mvc;
    @Autowired
    private ObjectMapper json;
    @Autowired
    private CreditService credits;
    @Autowired
    private CreditContractRepository contracts;
    @Autowired
    private CardRepository cards;
    @Autowired
    private MobilePaymentRepository payments;
    @Autowired
    private InMemoryLedger ledger;
    @MockitoBean
    private ClientApi clients;
    @MockitoBean
    private DepositApi deposits;

    private CreditIssued credit;
    private String card;
    private String pin;

    @BeforeEach
    void issueCreditWithCard() {
        payments.deleteAll();
        cards.deleteAll();
        contracts.deleteAll();
        ledger.reset();
        when(clients.get(anyLong())).thenAnswer(call ->
            new ClientInfo(call.getArgument(0), "Иванов Иван Иванович", "MP", "7654321", "3170590A077PB4"));
        credit = credits.open(new CreditRequest("K-000001", 1L, "BYN", 1L, new BigDecimal("1000.00"),
            new BigDecimal("17.65"), 6, START, START.plusMonths(6)));
        card = credit.card().cardNumber();
        pin = credit.card().pin();
    }
}

```

```

@Test
@DisplayName("Авторизация: верный PIN-код принимается")
void authorize() throws Exception {
    send("AUTHORIZE")
        .andExpect(jsonPath("$.status").value("OK"))
        .andExpect(jsonPath("$.data.holder").value("Иванов Иван Иванович"));
}

@Test
@DisplayName("Три неверных PIN-кода подряд блокируют карту; перевыпуск PIN-кода снимает блокировку")
void wrongPinThreeTimes() throws Exception {
    send(card, wrongPin(), "AUTHORIZE").andExpect(jsonPath("$.code").value("WRONG_PIN"))
        .andExpect(jsonPath("$.data.attemptsLeft").value(2));
    send(card, wrongPin(), "AUTHORIZE").andExpect(jsonPath("$.data.attemptsLeft").value(1));
    send(card, wrongPin(), "AUTHORIZE").andExpect(jsonPath("$.code").value("CARD_BLOCKED"));
    send("AUTHORIZE").andExpect(jsonPath("$.code").value("CARD_BLOCKED")); // верный PIN уже не помогает

    String response = mvc.perform(post("/api/credits/" + credit.contract().id() + "/pin"))
        .andExpect(status().isOk()).andReturn().getResponse().getContentAsString();
    pin = json.readTree(response).get("pin").asText();
    send("AUTHORIZE").andExpect(jsonPath("$.status").value("OK"));
}

@Test
@DisplayName("Счётчик неверных попыток сбрасывается после верного PIN-кода")
void attemptsAreResetAfterSuccess() throws Exception {
    send(card, wrongPin(), "AUTHORIZE").andExpect(jsonPath("$.data.attemptsLeft").value(2));
    send("AUTHORIZE").andExpect(jsonPath("$.status").value("OK"));
    send(card, wrongPin(), "AUTHORIZE").andExpect(jsonPath("$.data.attemptsLeft").value(2));
}

@Test
@DisplayName("Неизвестная карта и неполная транзакция отклоняются")
void unknownCardAndIncompleteTransaction() throws Exception {
    send("911238000009993", pin, "AUTHORIZE").andExpect(jsonPath("$.code").value("CARD_NOT_FOUND"));
    transaction(List.of(item("CARD", card), item("OPERATION", "BALANCE")))
        .andExpect(jsonPath("$.code").value("BAD_REQUEST"));
    send("TRANSFER").andExpect(jsonPath("$.code").value("UNKNOWN_OPERATION"));
    mvc.perform(post("/api/atm/transactions").contentType(MediaType.APPLICATION_JSON).content("{\"items\": []}"))
        .andExpect(status().isBadRequest());
}

@Test
@DisplayName("Просмотр остатка кредитного счёта")
void balance() throws Exception {
    send("BALANCE")
        .andExpect(jsonPath("$.status").value("OK"))
        .andExpect(jsonPath("$.data.balance").value(1000.00))
        .andExpect(jsonPath("$.data.currency").value("BYN"))
        .andExpect(jsonPath("$.data.account").value(credit.contract().mainAccount()))
        .andExpect(jsonPath("$.data.bankDate").value("2026-10-01"));
}

@Test
@DisplayName("Снятие наличных: проводки через кассу, остаток уменьшается")
void withdraw() throws Exception {
    send("WITHDRAW", item("AMOUNT", "300"))
        .andExpect(jsonPath("$.status").value("OK"))
        .andExpect(jsonPath("$.message").value("Заберите деньги"))
        .andExpect(jsonPath("$.data.amount").value(300))
        .andExpect(jsonPath("$.data.balance").value(700.00));

    String cash = ledger.systemAccount(LedgerApi.CASH, "BYN").number();
    assertThat(ledger.balance(credit.contract().mainAccount()).isEqualByComparingTo("700.00"));
    assertThat(ledger.debit(cash).isEqualByComparingTo("300.00"));
    assertThat(ledger.credit(cash).isEqualByComparingTo("300.00"));
    assertThat(ledger.operations("K-000001").extracting(OperationInfo::description)
        .containsExactly("Выделение кредита банком", "Перевод кредита в кассу", "Получение кредита через кассу"));
}

@Test
@DisplayName("Снятие суммы больше остатка — сообщение об ошибке, счёт не меняется")
void withdrawMoreThanAvailable() throws Exception {
    send("WITHDRAW", item("AMOUNT", "1001"))
        .andExpect(jsonPath("$.status").value("ERROR"))
        .andExpect(jsonPath("$.code").value("INSUFFICIENT_FUNDS"));

    assertThat(ledger.balance(credit.contract().mainAccount()).isEqualByComparingTo("1000.00"));
    // отказ не ломает последующие операции с той же картой
    send("WITHDRAW", item("AMOUNT", "1000")).andExpect(jsonPath("$.status").value("OK"));
}

```

```

@ParameterizedTest(name = "сумма «{0}»")
@ValueSource(strings = {"0", "-5", "abc", "10.5", "", "1e3", "12345678901"})
@DisplayName("Сумма снятия – целое положительное число")
void withdrawAmountIsValidated(String amount) throws Exception {
    send("WITHDRAW", item("AMOUNT", amount)).andExpect(jsonPath("$.code").value("INVALID_ID_AMOUNT"));

    assertThat(ledger.balance(credit.contract().mainAccount()).isEqualByComparingTo("1000.00"));
}

@Test
@DisplayName("Платёж за мобильную связь: деньги уходят на расчётный счёт оператора")
void payment() throws Exception {
    send("PAYMENT", item("OPERATOR", "MTS"), item("PHONE", "0297654321"), item("AMOUNT", "25.50"))
        .andExpect(jsonPath("$.status").value("OK"))
        .andExpect(jsonPath("$.data.operator").value("MTC"))
        .andExpect(jsonPath("$.data.phone").value("0297654321"))
        .andExpect(jsonPath("$.data.balance").value(974.50))
        .andExpect(jsonPath("$.data.reference").isString());

    AccountInfo operator = ledger.accounts("OPERATOR-MTS-BYN").get(0);
    assertThat(operator.chartCode()).isEqualTo("3012");
    assertThat(operator.balance()).isEqualByComparingTo("25.50");
    assertThat(ledger.balance(credit.contract().mainAccount()).isEqualByComparingTo("974.50"));
    assertThat(payments.findAll()).singleElement().satisfies(p -> {
        assertThat(p.getPhone()).isEqualTo("0297654321");
        assertThat(p.getAmount()).isEqualByComparingTo("25.50");
    });

    // второй платёж тому же оператору идёт на тот же счёт
    send("PAYMENT", item("OPERATOR", "MTS"), item("PHONE", "0331112233"), item("AMOUNT", "10"))
        .andExpect(jsonPath("$.status").value("OK"));
    assertThat(ledger.accounts("OPERATOR-MTS-BYN").hasSize(1));
    assertThat(ledger.balance(operator.number()).isEqualByComparingTo("35.50"));
}

@ParameterizedTest(name = "{0} / {1} / {2} -> {3}")
@CsvSource({
    "BEELINE, 0297654321, 10, UNKNOWN_OPERATOR",
    "A1, 297654321, 10, INVALID_PHONE",
    "A1, 0177654321, 10, INVALID_PHONE",
    "A1, 02976543210, 10, INVALID_PHONE",
    "A1, 0297654321, 0, INVALID_AMOUNT",
    "A1, 0297654321, 10.555, INVALID_AMOUNT",
    "A1, 0297654321, 1000.01, INSUFFICIENT_FUNDS",
})
@DisplayName("Платёж проверяется: оператор, номер из 10 цифр, сумма, достаточность средств")
void paymentIsValidated(String operator, String phone, String amount, String code) throws Exception {
    send("PAYMENT", item("OPERATOR", operator), item("PHONE", phone), item("AMOUNT", amount))
        .andExpect(jsonPath("$.status").value("ERROR"))
        .andExpect(jsonPath("$.code").value(code));

    assertThat(ledger.balance(credit.contract().mainAccount()).isEqualByComparingTo("1000.00"));
    assertThat(payments.count()).isZero();
}

@Test
@DisplayName("Остаток депозитных счетов запрашивается у сервиса депозитов")
void depositBalance() throws Exception {
    when(deposits.byClient(1L)).thenReturn(List.of(
        new DepositInfo("Д-000001", "Свободное накопление", "BYN", new BigDecimal("1500.00"), "ACTIVE"),
        new DepositInfo("Д-000002", "Online-решение", "BYN", new BigDecimal("900.00"), "CLOSED")));

    send("DEPOSIT_BALANCE")
        .andExpect(jsonPath("$.status").value("OK"))
        .andExpect(jsonPath("$.data.deposits", hasSize(1)))
        .andExpect(jsonPath("$.data.deposits[0].number").value("Д-000001"))
        .andExpect(jsonPath("$.data.deposits[0].amount").value(1500.00));
}

@Test
@DisplayName("Сервис депозитов недоступен – банкомат получает отказ, а не ошибку сервера")
void depositServiceIsDown() throws Exception {
    when(deposits.byClient(1L)).thenThrow(new ResourceAccessException("Connection refused"));

    send("DEPOSIT_BALANCE")
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.code").value("SERVICE_UNAVAILABLE"));
}

@Test
@DisplayName("Справочник операторов связи")
void operators() throws Exception {

```

```

        mvc.perform(get("/api/atm/operators")).andExpect(jsonPath("$", hasSize(3)));
    }

    // ----- сборка транзакции банкомата -----

    private ResultActions send(String operation, Map<String, String>... extra) throws Exception {
        return send(card, pin, operation, extra);
    }

    @SafeVarargs
    private ResultActions send(String cardNumber, String pinCode, String operation, Map<String, String>... extra) throws Exception
    {
        List<Map<String, String>> items = new ArrayList<>(List.of(
            item("CARD", cardNumber), item("PIN", pinCode), item("OPERATION", operation)));
        items.addAll(List.of(extra));
        return transaction(items);
    }

    private ResultActions transaction(List<Map<String, String>> items) throws Exception {
        return mvc.perform(post("/api/atm/transactions").contentType(MediaType.APPLICATION_JSON)
            .content(json.writeValueAsString(Map.of("items", items))))
            .andExpect(status().isOk());
    }

    private static Map<String, String> item(String name, String value) {
        return Map.of("name", name, "value", value);
    }

    private String wrongPin() {
        return "0000".equals(pin) ? "1111" : "0000";
    }
}

```

Листинг А.36 – credit-service/src/test/java/by/bsuir/bank/credit/CardNumbersTest.java

```

package by.bsuir.bank.credit;

import by.bsuir.bank.credit.domain.Card;
import by.bsuir.bank.credit.domain.CardNumbers;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;

import static org.assertj.core.api.Assertions.assertThat;

/** Модульные тесты номера карты (алгоритм Луна) и счётчика неверных PIN-кодов. */
class CardNumbersTest {

    @ParameterizedTest
    @ValueSource(strings = {"4539148803436467", "4111111111111111", "5500000000000004"})
    void acceptsKnownValidNumbers(String number) {
        assertThat(CardNumbers.isValid(number)).isTrue();
    }

    @ParameterizedTest
    @ValueSource(strings = {"4539148803436468", "4111111111111111", "4111111111111111", "4111-1111-1111-1111", ""})
    void rejectsWrongNumbers(String number) {
        assertThat(CardNumbers.isValid(number)).isFalse();
    }

    @Test
    void generatedNumbersAreValidAndUnique() {
        String first = CardNumbers.generate(1);
        String second = CardNumbers.generate(2);

        assertThat(first).hasSize(16).startsWith("911238").isNotEqualTo(second);
        assertThat(CardNumbers.isValid(first)).isTrue();
        assertThat(CardNumbers.isValid(second)).isTrue();
        assertThat(CardNumbers.isValid(null)).isFalse();
    }

    @Test
    void cardIsBlockedAfterThirdWrongPin() {
        Card card = new Card();

        assertThat(card.registerFailedAttempt()).isEqualTo(2);
        assertThat(card.registerFailedAttempt()).isEqualTo(1);
        assertThat(card.isBlocked()).isFalse();
        assertThat(card.registerFailedAttempt()).isZero();
    }
}

```

```

        assertThat(card.isBlocked()).isTrue();

        card.resetAttempts();
        assertThat(card.isBlocked()).isFalse();
        assertThat(card.getFailedAttempts()).isZero();
    }
}

```

Запуск сервисов

Листинг A.37 – scripts/run-all.sh

```

#!/usr/bin/env bash
# Запуск всех микросервисов банка. Логи пишутся в logs/, остановка – scripts/stop-all.sh
# Использование: scripts/run-all.sh [client account deposit credit atm]
set -euo pipefail
cd "$(dirname "$0")/.."

services=("@")
if [ ${#services[@]} -eq 0 ]; then
    services=(client account deposit credit atm)
fi

missing=0
for name in "${services[@]}; do
    [ -f "$name-service/target/$name-service-1.0.0.jar" ] || missing=1
done
if [ "$missing" -eq 1 ]; then
    ./mvnw -q -DskipTests package
fi

mkdir -p logs
port_of() {
    case "$1" in client) echo 8081 ;; account) echo 8082 ;; deposit) echo 8083 ;; credit) echo 8084 ;; atm) echo 8085 ;; esac
}

for name in "${services[@]}; do
    nohup java -jar "$name-service/target/$name-service-1.0.0.jar" > "logs/$name.log" 2>&1 &
    echo $! > "logs/$name.pid"
done

for name in "${services[@]}; do
    port=$(port_of "$name")
    for _ in $(seq 1 90); do
        curl -s -o /dev/null "http://localhost:$port/" && break
        sleep 1
    done
    echo "$name-service: http://localhost:$port/"
done

```

Листинг A.38 – scripts/stop-all.sh

```

#!/usr/bin/env bash
# Остановка микросервисов, запущенных scripts/run-all.sh
cd "$(dirname "$0")/.."
for pid_file in logs/*.pid; do
    [ -f "$pid_file" ] || continue
    kill "$(cat "$pid_file")" 2>/dev/null || true
    rm -f "$pid_file"
done

```